软件系统综合课程设计——

软件质量/缺陷



系统功能测试应该考虑哪些问题？

- 这项功能和哪些业务有关系呢？
- 这项功能在系统中起什么作用？
- 谁会使用使用这项功能？
- 用户如何使用这项功能？
- 这些功能的典型应用场景有哪些？
- 掌握这个功能的相关信息清楚吗？充
- 这个功能和哪些功能相关吗？
- ……

1) 这个产品的愿景是什么？
2) 这个产品是免费产品吗？
3) 这个产品的用户特点是什么？目前有多少用户？
4) 它所对应的竞争性产品（竞品）有哪些？
5) 这个产品只局限于中文版吗？
6) 即使是中文版，会让港澳、台湾地区用户使用吗？
7) 这个项目的测试周期多长？哪几个人参与测试？
8) 这个产品开发平台是什么？运用了哪些软件技术？
9) 这个产品运行在什么样的环境上？
10) 开发有没有做过单元测试？
11) 单元测试报告在哪里？单元测试覆盖度率如何？
12) ……

软件测试执行与跟踪

1、测试执行过程要点

（1）不同测试阶段的执行要点

测试人员参与需求与设计评审

程序员负责单元测试，完成反馈给测试组

尽早考虑集成测试，彻底完成模块之间各个接口测试。

功能测试：从用户角度，落实使用场景、边界和特殊场合缺陷。

专项测试：考虑实际运行场景

软件测试执行与跟踪

1、测试执行过程要点

（2）测试用例执行

跟踪测试执行

（3）团队建设与沟通

领导及服务好团队（培训 指导、解决冲突、提供资源、建立项目目标和优先级等）

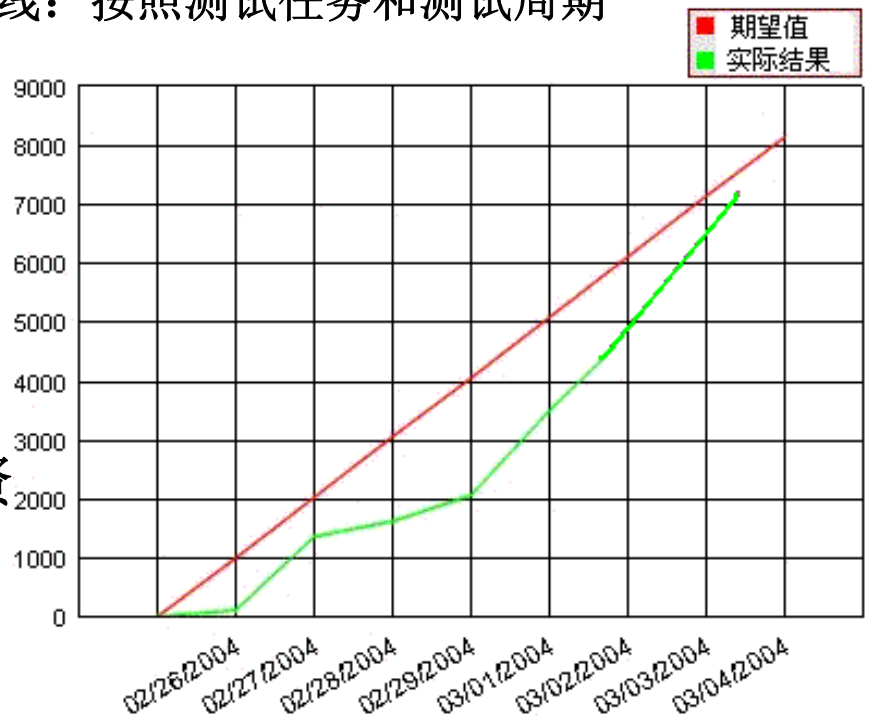
（4）测试执行结束

全过程、全方位审视、检查测试计划 用例执行到位

分析：缺陷进行逐个分析，了解对产品影响的程度，从而决定是否中止测试

总结和分析： 测试结果分析、质量进行评估、编制测试报告（签字），优化改进

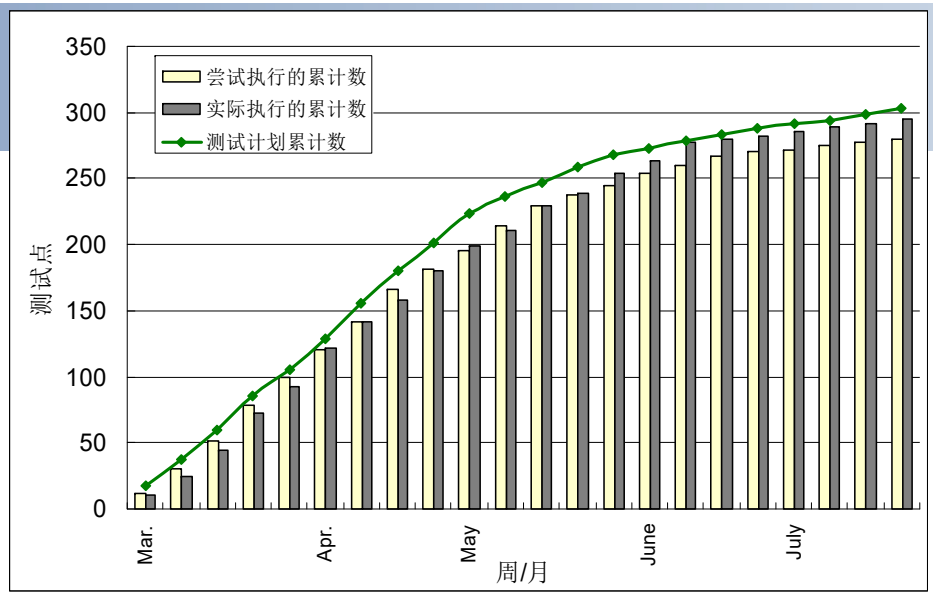
测试执行情况跟踪曲线：按照测试任务和测试周期



2、测试项目进度的管理方法

（1）测试进度S曲线法

三阶段：初始、紧张、成熟阶段



进度S曲线法通过对计划中、尝试的与实际的进度三者对比来实现的，其采用的基本数据主要是测试用例或测试点的数量随时间推移，曲线形状像S。其中紧张阶段强度高于初始+成熟阶段。

NOB，Number of Open Bug

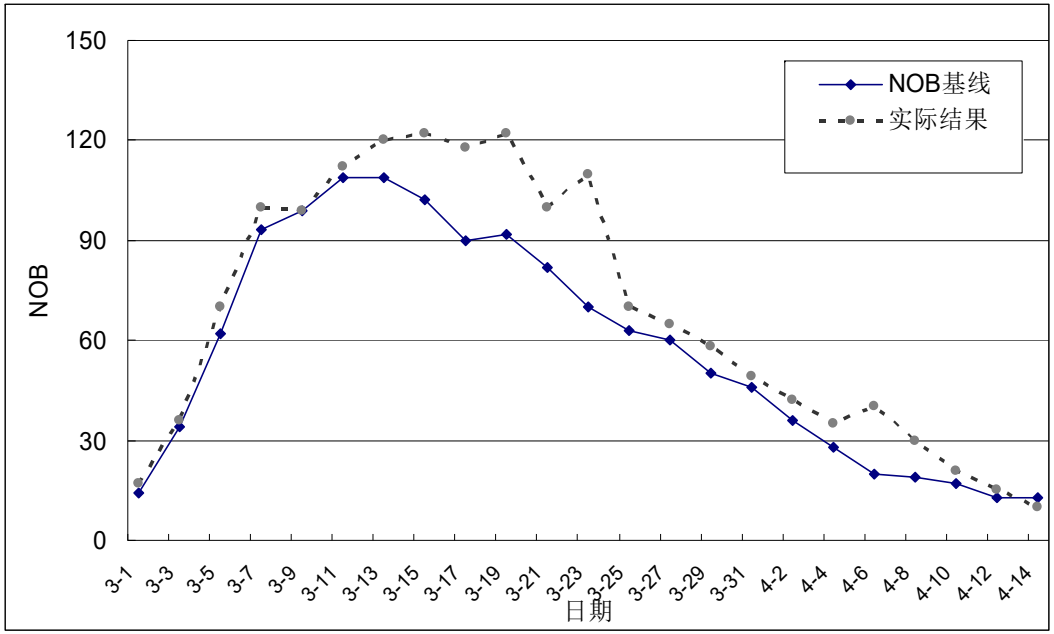
2、测试项目进度的管理方法

（1）测试进度NOB曲线法

一定程度反映了软件质量和测试进度随时间的发展趋势。

基线： 测试进度设计曲线或者理想曲线，什么时候高峰，发布前降到足够低。

考虑潜在的缺陷逃逸或者发现延迟。



测试反直觉理论: 发现缺陷越多的地方，有更多缺陷将发现

3、测试过程的管理工具

（1）MeterSphere ： https://metersphere.io/

是一站式开源持续测试平台，涵盖测试管理、接口测试、性能测试、团队协作等功能



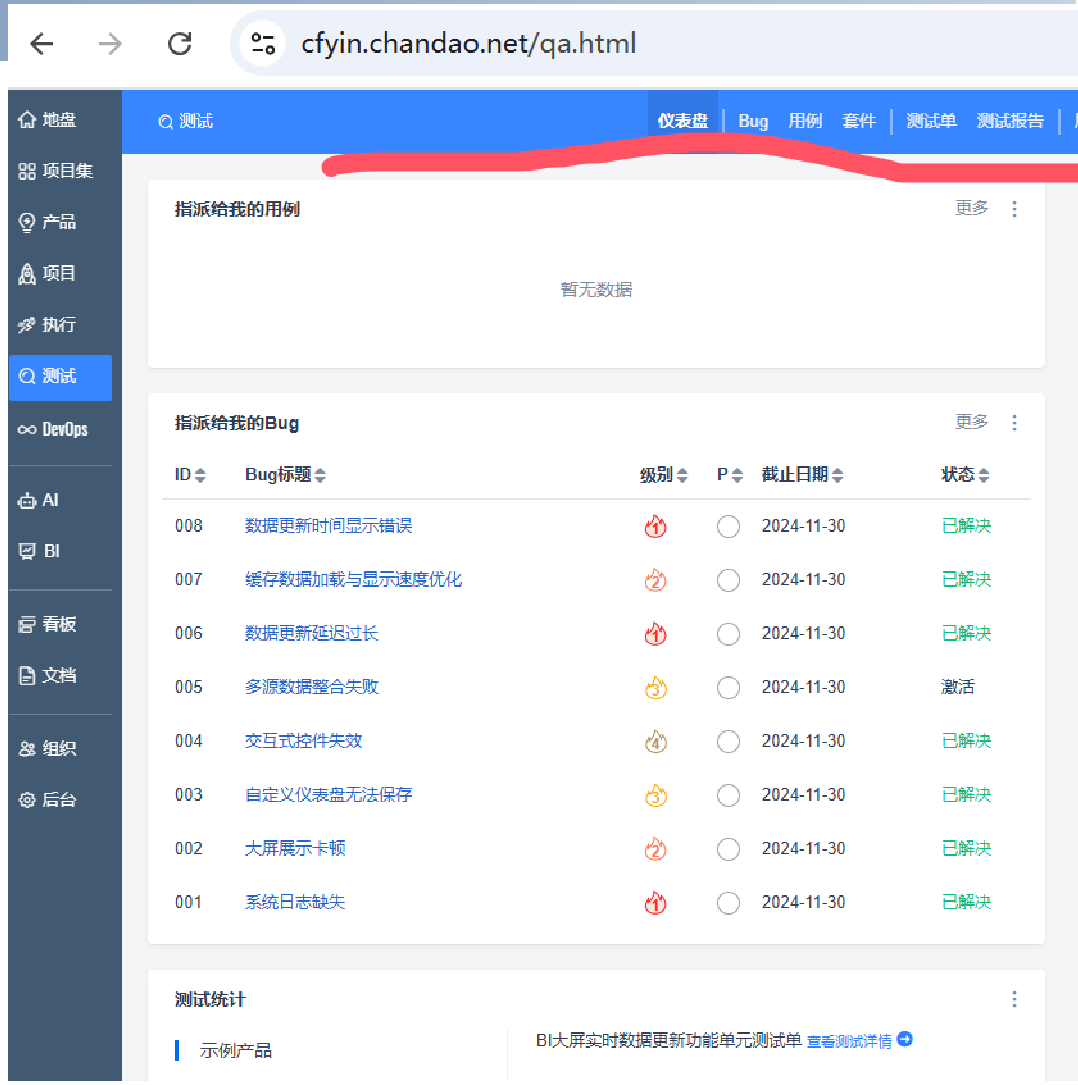
软件测试执行与跟踪

3、测试过程的管理工具

（2）禅道

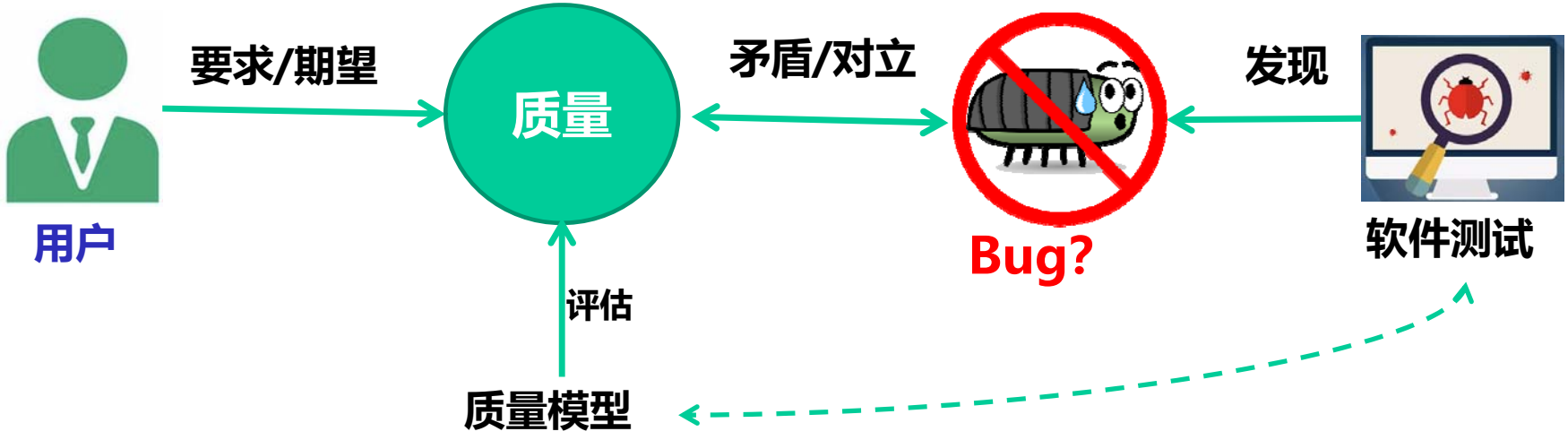
禅道是一款集产品管理、项目管理、质量管理于一体的开源测试管理工具，支持需求跟踪、缺陷管理、测试用例维护及自动化测试等功能。

https://www.chandao.net/qa.html



缺陷是质量的对立面

要了解什么是缺陷(defect)，就必须清楚“质量(Quality)”概念，因为缺陷是相对质量而存在的，违背了质量、违背了客户的意愿，不能满足客户的要求，就会引起缺陷或产生缺陷



质量的定义

ISO 8492关于质量的定义：“ 产品或者服务满足明示或者暗示需求能力的固有特性和特征集合。”

质量的3个维度：

（1）符合目标： 目标是客户提出的，符合目标就是判断我们是不是在做我们需要做的事情。在这个维度上认为的质量是“产品是否按照客户的需要运行”。

（2）符合需求： 检验产品是否在做让它做的事情。 软件产品的需求必须是可度量的开发者而言，质量就是满足用户的需求或规格说明书

（3）符合实际需求： 实际的需求包括用户明确说明的和隐含的需求，产品是为用户提供服务的。

质量的定义

“客户”和“用户”的区别：

客户之真正能够决定是否购买你软件的人，而用户是实际使用软件的人。

我们开发软件是应该更多地站在用户的角度去开发，要做到让用户用得方便、舒服、提高了效率。用户的意见能直接反馈到客户那里，作为客户选购软件的依据。

了解两者的区别，在分析需求和进行产品的质量验证时可以做出不同的权衡。

质量的定义

没有缺陷的产品和服务是不可能的。但是控制在一定级别的缺陷数量是正常的，是可以接受的。

经济性：质量经常是和成本紧密联系在一起的，一个高质量的产品同时也意味着高投入。这是设计的质量和一致性质量的一个矛盾。

高的质量要求：

（1）需求规格说明书足够详细：这些规格说明书可进行定量的分析。然而许多组织没有能力或者不愿意制作如此详细的规格说明书。

（2）遵循良好定义的标准和过程：技术人员认为规范和标准会束缚他们的创造力，因而不愿意遵照标准做事。

软件质量（Software Quality）的概念

软件质量是软件符合明确叙述的功能和性能需求、文档中明确描述的开发标准、以及所有专业开发的软件都应具有的隐含特征的程度。



软件质量模型——ISO/IEC 9126

新的ISO/IEC 9126软件质量模型，软件质量包括“内部质量”、“外部质量”和“使用质量”三部分。

（1）内部质量:从内部观点出发的软件产品特性的总体，是针对内部质量需求被测量和评价的质量。

内部质量特征主要包括：可维护性、灵活性、可移植性、可重用性、可读性、可测试性、可理解性。

（2）外部质量：测试其外部属性，即代码执行时的行为。

外部质量特征主要包括：正确性、可用性、效率、可靠性、完整性、适应性、精确性、坚固性。

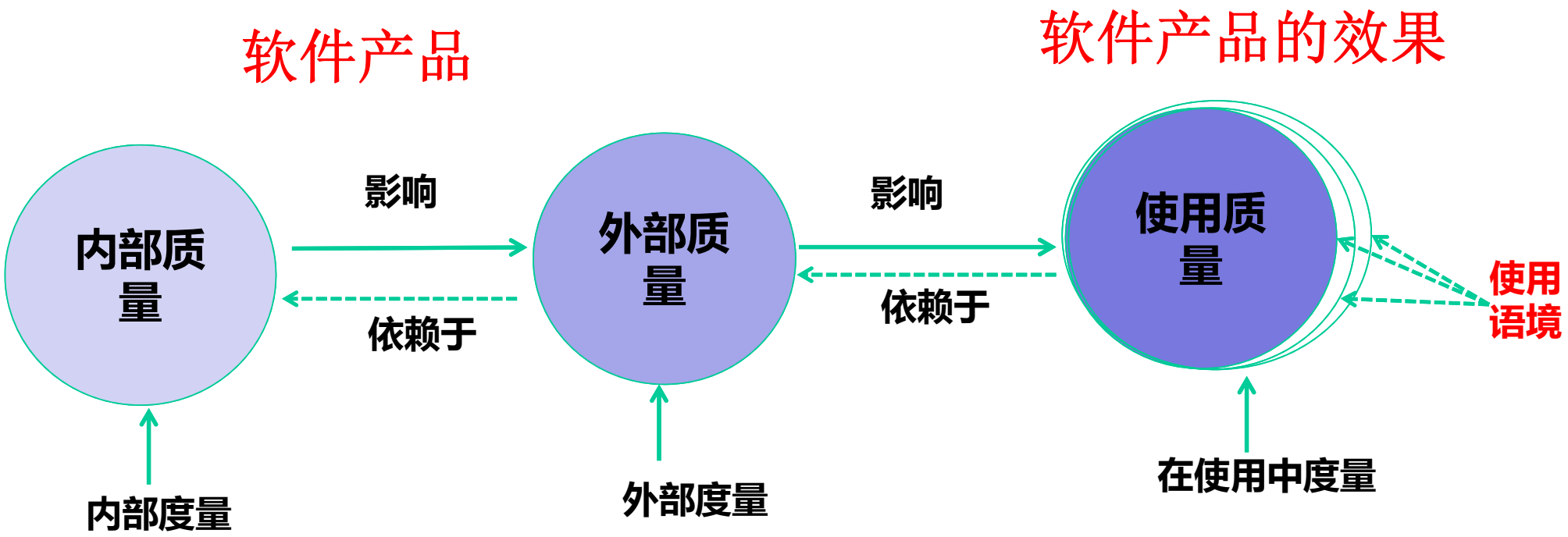
软件质量模型——ISO/IEC 9126

（3）使用质量：在规定的使用环境下软件产品使特定用户在达到规定目标方面的能力。

从用户观点出发，来看待软件产品用于特定环境和条件下的质量，反映的是从用户角度看到的软件产品在适当系统环境下满足其需求的程度。

使用质量质量特征:有效性、生产率、安全性、满意程度等。

内部质量→外部质量→使用质量



软件质量模型

质量模型是指提供声明质量需求和评价质量基础的特性以及特性之间关系的集合。换句话说质量模型是用来描述质量需求以及对质量进行评价的理论基础。对于软件质量进行评判需要一个对评判工作有指导意义的质量模型。

（1）可以帮助开发人员开发出符合标准的软件产品

（2）可以帮助评判人员识别可能影响软件质量的风险。

软件的质量标准主要分为两个方向： 产品质量和过程质量。

（1）产品质量：通过测量获得产品质量特性的有关数据，确定是否满足规定。

模型：**ISO/IEC 25010** 软件质量模型（**2011~2023**年）

（2）过程质量：评判产生产品的过程的好坏，度量软件进程，成本、质量、变化 风险。

模型：**CMM** **ISO 9000**

产品质量模型

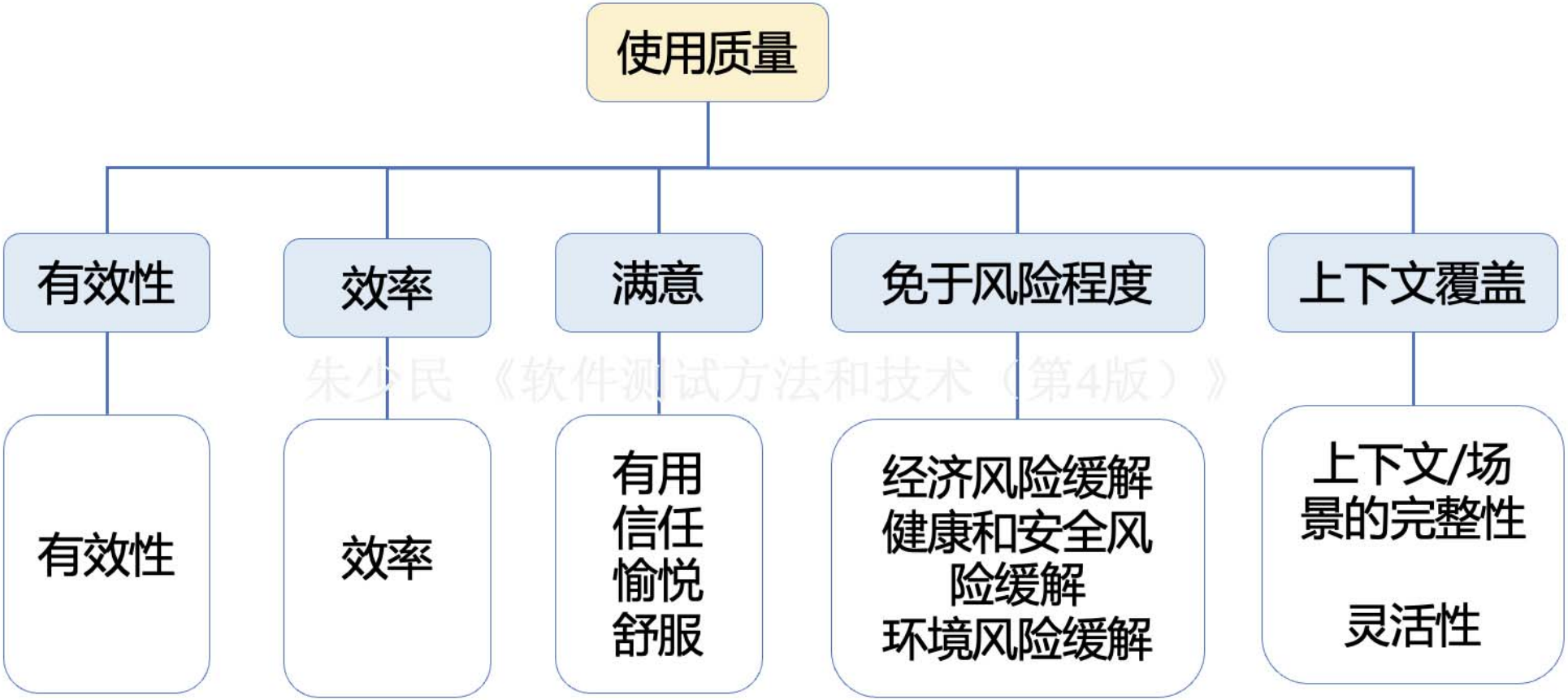
质量模型ISO/IEC 25010:2023 将软件质量划分为 9 个主要特性，每个特性进一步细化为子特性，形成多层次的评估体系。

质量特性	子特性	定义与示例
1. 功能适用性	功能完整性、正确性、适当性	确保功能覆盖用户需求（如金融软件精确计算利息）。
2. 性能效率	时间特性、资源利用率、容量	高并发系统的响应速度（如电商秒杀活动支持百万级用户）。
3. 兼容性	共存性、互操作性	多系统共享资源互不干扰（如智能家居设备跨平台联动）。
4. 交互能力	包容性、可学习性、用户粘性、容错性	支持无障碍设计（如视障用户通过屏幕阅读器操作软件）。

产品质量模型-质量模型ISO/IEC 25010:2023

质量特性	子特性	定义与示例
5. 可靠性	成熟性、容错性、易恢复性、可用性	系统故障后快速恢复（如云服务的自动故障转移机制）。
6. 安全性	保密性、完整性、不可抵赖性、真实性、可核查性	数据加密与权限控制（如医疗系统采用区块链技术保护患者隐私）。
7. 可维护性	模块化、易分析性、可复用性	代码低耦合设计（如微服务架构支持独立模块升级）。
8. 灵活性	适应性、易安装性、可扩展性	跨平台兼容（如容器化技术实现“一次构建，随处运行”）。
9. 无害性（新增）	风险识别、故障安全、安全集成	高风险系统自动停机保护（如核电站传感器触发紧急关闭）。

使用质量模型



朱少民《软件测试方法和技术（第4版）》

软件过程能力成熟度模型CMM

CMM（能力成熟度模型，**Capability Maturity Model**）是由美国卡内基梅隆大学软件工程研究所（**SEI**）提出的软件过程评估框架，旨在通过分级改进提升软件开发组织的质量管理能力。

是国际公认的规格最高、难度最大、最具权威性的软件能力评估体系，被誉为软件国际化的“通行证”，具有极高的含金量。

CMMI认证共有**5**个等级，对应软件企业能力成熟度的五个层次，**CMMI 5**级即为该认证体系的最高等级。

软件能力成熟度集成模型CMMI Capability Maturity Model Integration

由美国国防部与卡内基-梅隆大学和美国国防工业协会共同开发和研制的。

CMMI本质是软件管理工程的一个部分，**软件评估标准**。

CMMI认证的目的是帮助软件企业对软件工程过程进行管理和改进，增强开发与改进能力，从而能按时地、不超预算地开发出**高质量的软件**。

CMMI主要关注点就是成本效益、明确重点、过程集中和灵活性四个方面。

1987年开始认证，最权威认证体系，5个等级，18个过程域，300多个关键实践。

软件能力成熟度集成模型CMMI Capability Maturity Model Integration

当今软件企业的核心竞争力已从单纯的技术能力转向系统性成熟度的较量。无论是参与国际竞标、提升客户信任度，还是优化内部管理流程，CMMI(软件能力成熟度集成模型)认证已成为全球软件企业公认的“黄金通行证”。

CMMI5级企业

全球：仅6.6%企业通过CMMI V3.0五级认证。

中国：红海云连续三次通过CMMI5级，上海网钜获“双5”认证（CMMI5+TMMi5）

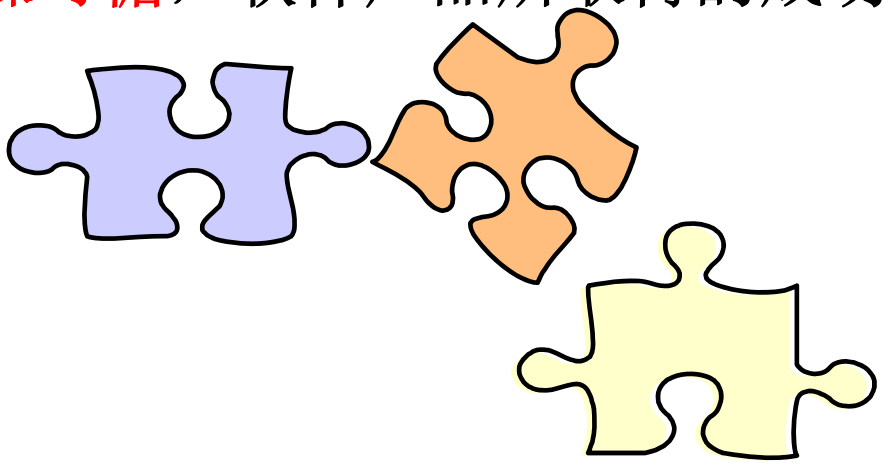


CMM五级模型

第一级：初始级

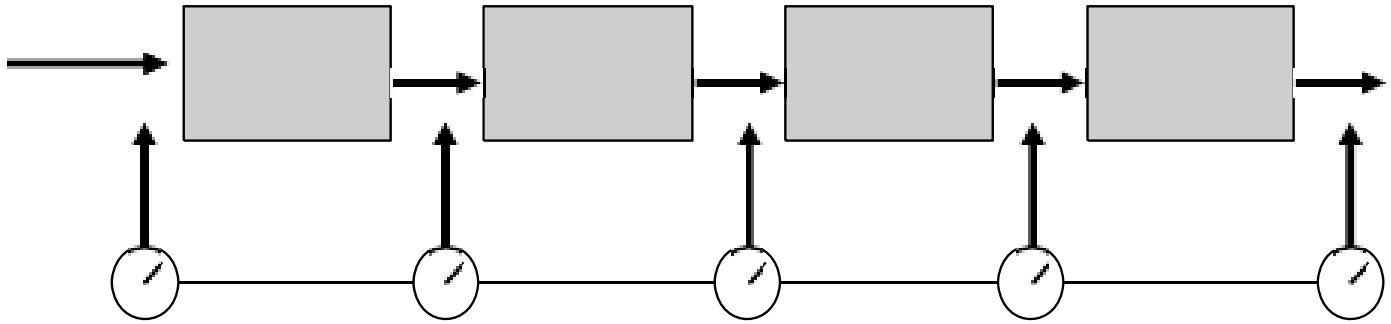


特点是软件过程无秩序，有时甚至是混乱的。软件过程定义几乎没有章法和步骤可循，软件产品所取得的成功往往依赖极个别人的努力和机遇。



CMM五级模型

第二级：可重复级



已建立了基本的项目管理过程，可用于对成本，进度和功能特性进行跟踪。对类似的应用项目，有章可循，并能重复以往所取得的成功。



CMM五级模型

第二级: 可重复级

按计划开发软件开发的首要问题不是技术问题而是管理问题。

可重复级的焦点集中在软件管理过程上。一个可管理的过程则是一个可重复级的过程，一个可重复级的过程则能逐渐进化和成熟。

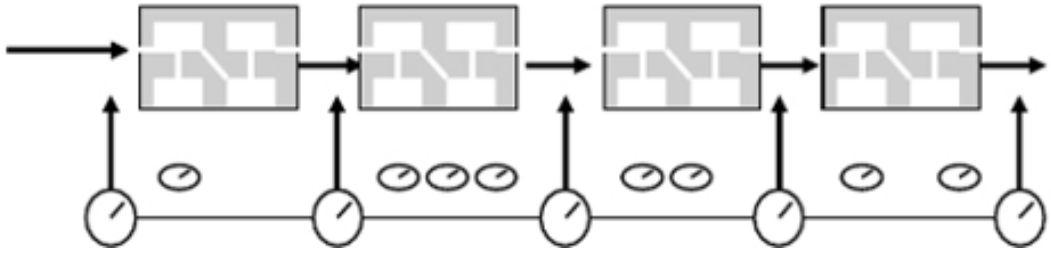
该级管理过程包括了需求管理（RM）、软件项目计划（SPP）软件项目跟踪和监控管理(SPTO)、软件质量保证（SQA）、配置管理(SCM)和子合同管理(SMM)五个方面。

通过实施这些过程，从管理角度可以看到一个按计划执行的且阶段可控的软件开发过程。

6个关键过程域：主要涉及及建立软件项目管理控制

CMM五级模型

第三级：定义级



用于管理的、工程的软件过程均已实现文档化、标准化，并形成了整个软件组织的标准软件过程。全部项目均已采用与实际情况相吻合的、适当修改的标准软件过程来进行。

3级7个过程域，满足3级认证，需要满足2，3级所有过程域。

CMMI 3级有18个过程域。

如：测试过程来说，就是先确定测试范围，测试类型，测试工具，测试环境，测试的准入准出条件等方面。

CMM五级模型

第三级：定义级

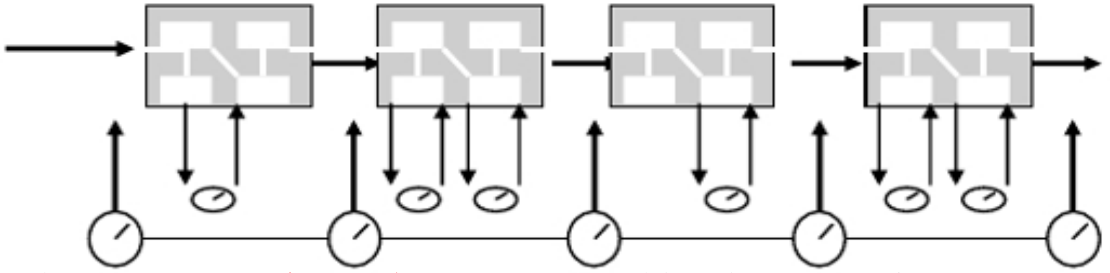
每个阶段的内部活动可见标准过程和项目定义过程裁剪，制定企业范围的工程化标准将这些标准集成到企业软件开发标准过程中去。

所有开发的项目需根据这个标准过程，剪裁出该项目的过程，并执行这些过程。对用于软件开发和维护的标准过程要以文件形式固定下来。

针对各个基本过程建立起文件化的“标准软件过程”较普遍的看法是，只有当达到了第3级能力成熟度时，才表明这个软件组织的软件能力“成熟”了。

CMM五级模型

第四级：已管理级



软件过程和产品质量有详细的度量标准，软件过程和产品质量得到了定量的认证和控制。

第4级除第2、3级所涵盖的18个流程领域外，

19. OPP：（Organizational Process Performance）组织过程性能。建立与维护组织过程性能的量化标准，以便使用量化方式的管理项目。

20. QPM（Quantitative Project Management) 量化的项目管理，量化管理项目已定义的项目过程，以达成项目既定的质量和过程性能目标。

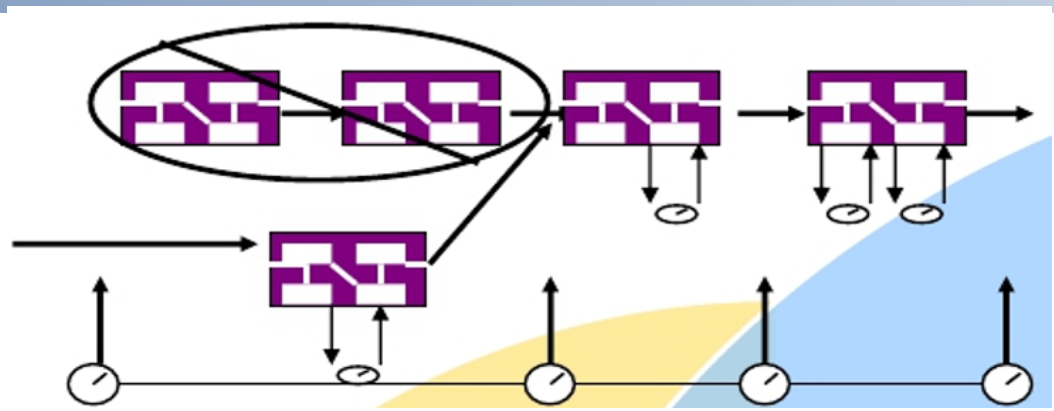
CMM五级模型

第四级：已管理级

过程可度量，预测值与结果之间的偏差可控所有过程都需建立相应的度量方式，所有产品的质量(包括工作产品和提交给用户的产品)需有明确的度量指标。这些度量是详尽的，且可用于理解、控制软件过程和产品，这种量化控制将使软件开发真正变成为工业生产活动。

处于这一级的组织已经能够为软件产品和软件过程设定定量的质量目标，并且能对跨项目的重要软件过程活动的效率和质量予以度量。

第五级：（不断）优化级



通过对来自过程、新概念和新技术等方面各种有用信息的定量分析，能够不断地、持续性的对过程进行改进。

第5级包含第2级到第4级的20个流程领域外

21.OID：(Organizational Innovation and Deployment）组织的创新与推展，选择并推展渐进创新的组织过程和技术改善，改善应是可度量的。

22. CAR：（Causal Analysis and Resolution）因果分析与解决。识别缺失的原因并进行矫正，进一步的防止未来再次发生。

等级	特征与要求	说明
初始级	过程无序，进度、预算、功能、质量不可预测，企业一般不具备稳定的软件开发环境、通常在遇到问的时候，就放弃原定的计划而只专注于编程与测试。	不需要认证
可重复级	建立了管理软件项目的政策，以及为贯彻执行这些政策而定的措施。基于以往项目的经验来计划与管理新的项目。达到此级别的企业过程已制度化，有纪律，可重复。	
定义级	过程实现标准化。有关软件工程和管理工程的特定的、面对整个企业的软件开发与维护的过程的文件将被制订出来。同时这些过程是集成到一个协调的整体。	
管理级	企业对产品及过程建立起定量的质量目标，同时在过程中加入规定的很清楚的连续的度量。作为企业的度量方案，要对项目的重要过程活动进行生产率和质量的度量。软件产品因此而具有可预期的高质量。达到该级的企业已实现过程定量化。	
优化级	整个企业将会把重点放在对过程进行不断的优化，采取主动的措施去找出过程的弱点与长处，以达到预防缺陷的目标，同时，分析各有关过程的有效性资料，作出对新技术的成本与效益的分析，并提出对过程进行修改的建议。达到该级的公司可自发的不断改进，防止同类缺陷二次出现。	

CMMI的价值

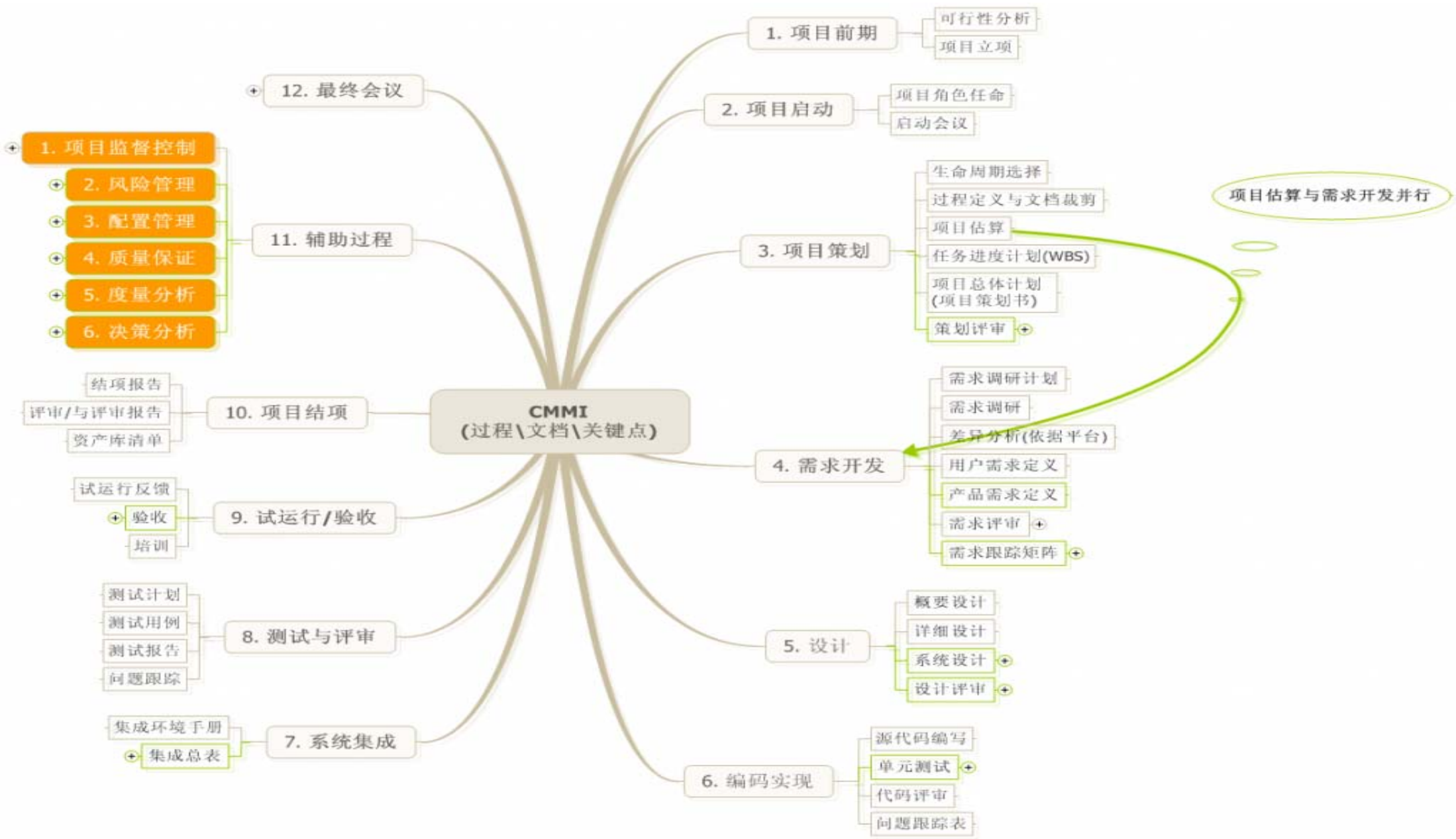
（1）能保证软件开发的质量与进度，能对项目开发过程进行规范。

（2）有利于成本控制。因为质量有所保证，浪费在修改、解决客户的抱怨方面的成本会降低很多。

（3）有助于提高软件开发者的职业素养。做事方法逐渐变得标准化、规范化。

（4）能够解决人员流动所带来的问题。公司通过过程改进，建立了财富库以共享经验。

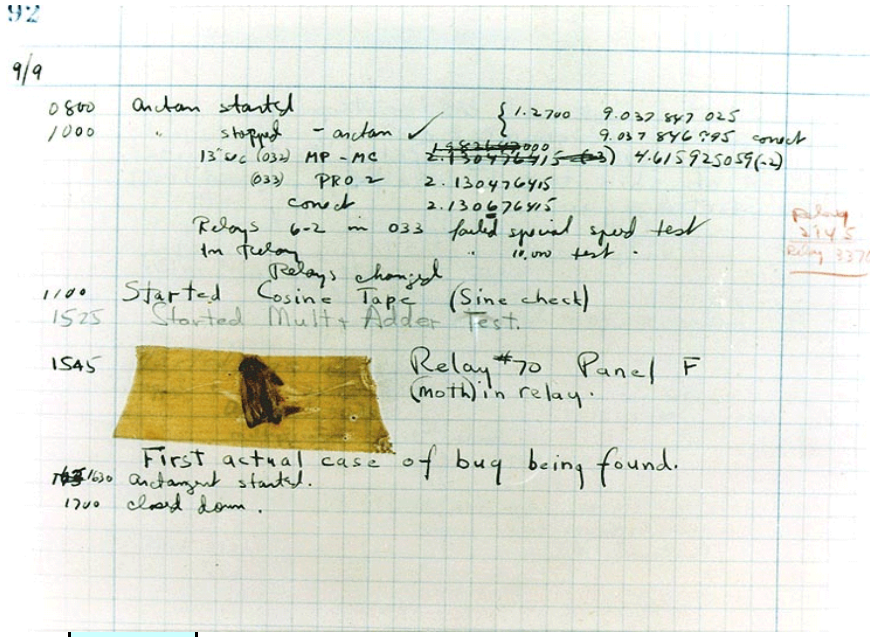
（5）有利于提升公司和员工绩效管理水平，以持续改进效益。通过度量和分析开发过程和产品，建立公司的效率指标（KPA）。

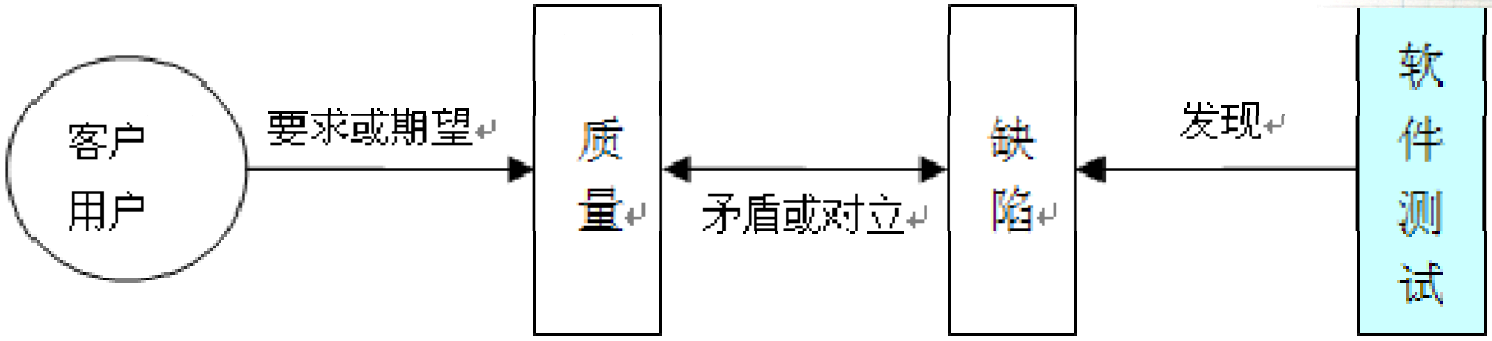

CMMI
(过程\文档\关键点)
1. 项目前期
可行性分析
项目立项
2. 项目启动
项目角色任命
启动会议
3. 项目策划
生命周期选择
过程定义与文档裁剪
项目估算
任务进度计划(WBS)
项目总体计划
(项目策划书)
策划评审
项目估算与需求开发并行
4. 需求开发
需求调研计划
需求调研
差异分析(依据平台)
用户需求定义
产品需求定义
需求评审
需求跟踪矩阵
5. 设计
概要设计
详细设计
系统设计
设计评审
6. 编码实现
源代码编写
单元测试
代码评审
问题跟踪表
7. 系统集成
集成环境手册
集成总表
8. 测试与评审
测试计划
测试用例
测试报告
问题跟踪
9. 试运行/验收
试运行反馈
验收
培训
10. 项目结项
结项报告
评审/与评审报告
资产库清单
11. 辅助过程
1. 项目监督控制
2. 风险管理
3. 配置管理
4. 质量保证
5. 度量分析
6. 决策分析
12. 最终会议

软件缺陷（IEEE国际标准729）

从产品内部看：软件缺陷是软件产品开发或维护过程中所存在的错误、毛病等各种问题；

从产品外部看：软件缺陷是系统所需要实现的某种功能的失效或违背。





Ron Patton 《Software Testing》

（1）软件测试员认为软件难以理解、不易使用、运行速度缓慢，或者最终用户认为不好。

（2）软件未达到需求规格说明书（SRS)中指明的功能。（显性需求）

（3）软件未达到需求规格说明书中虽未指出但应达到的目标。（隐形需求）

（4）软件出现了需求规格说明书中指明不会出现的错误。

（5）软件功能超出需求规格说明书中指明的范围。

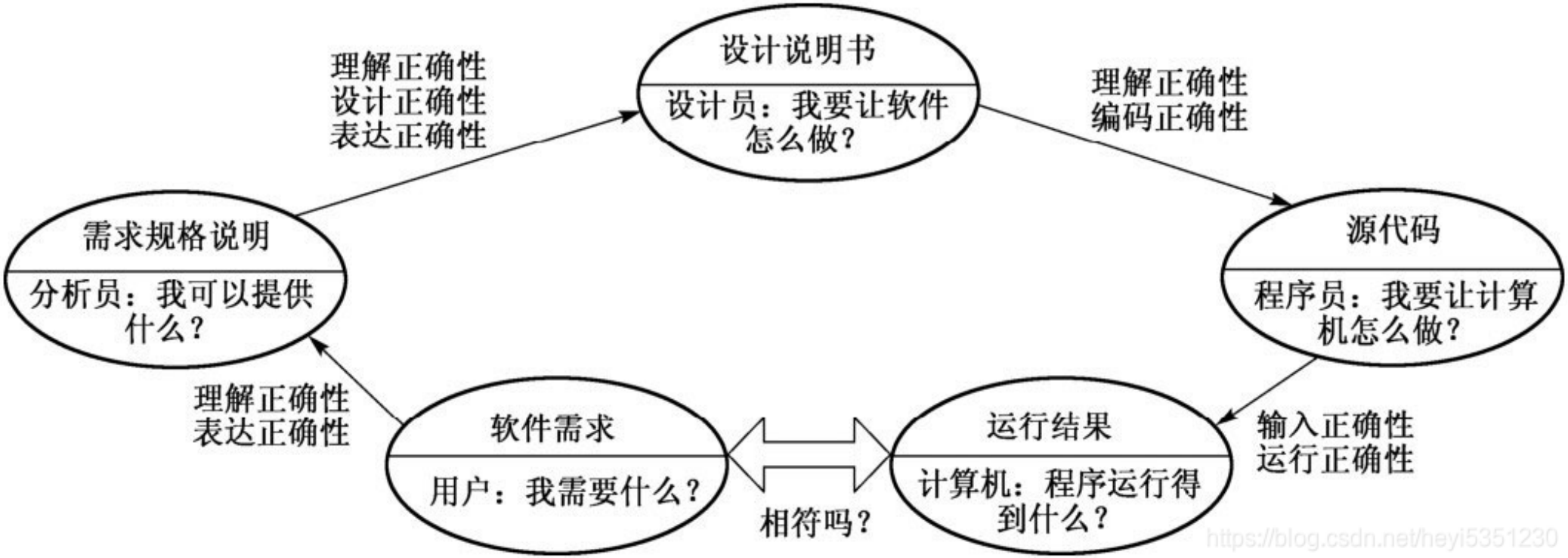
问题 ： （1）手机，屏幕显示的数字过小

（2）手机APP点击“提交”无反应

（3）手机突然解锁；

（4）手机某小程序可以实现扫描功能，设计说明书没有此功能

缺陷产生原因



软件产品全生命周期： 由于技术、软件及团队沟通等，不可避免会出现错误

Ron Patton 《Software Testing》

技术原因

（1）算法错误，语法错误，计算和精度问题，接口参数传递不匹配

团队问题

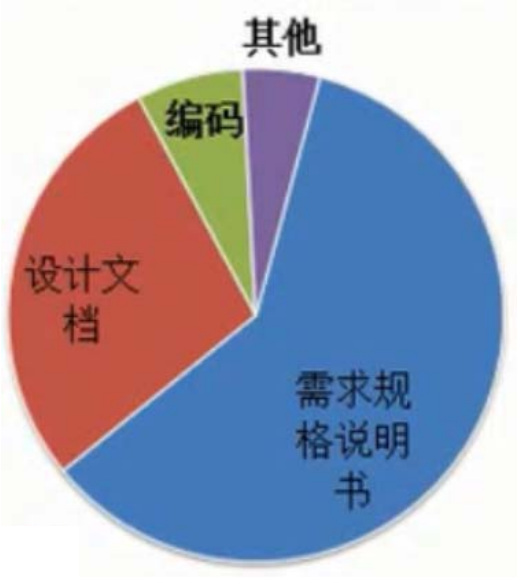
（1）误解、沟通不充分

软件问题

（1）文档错误、用户使用场合(user scenario)，

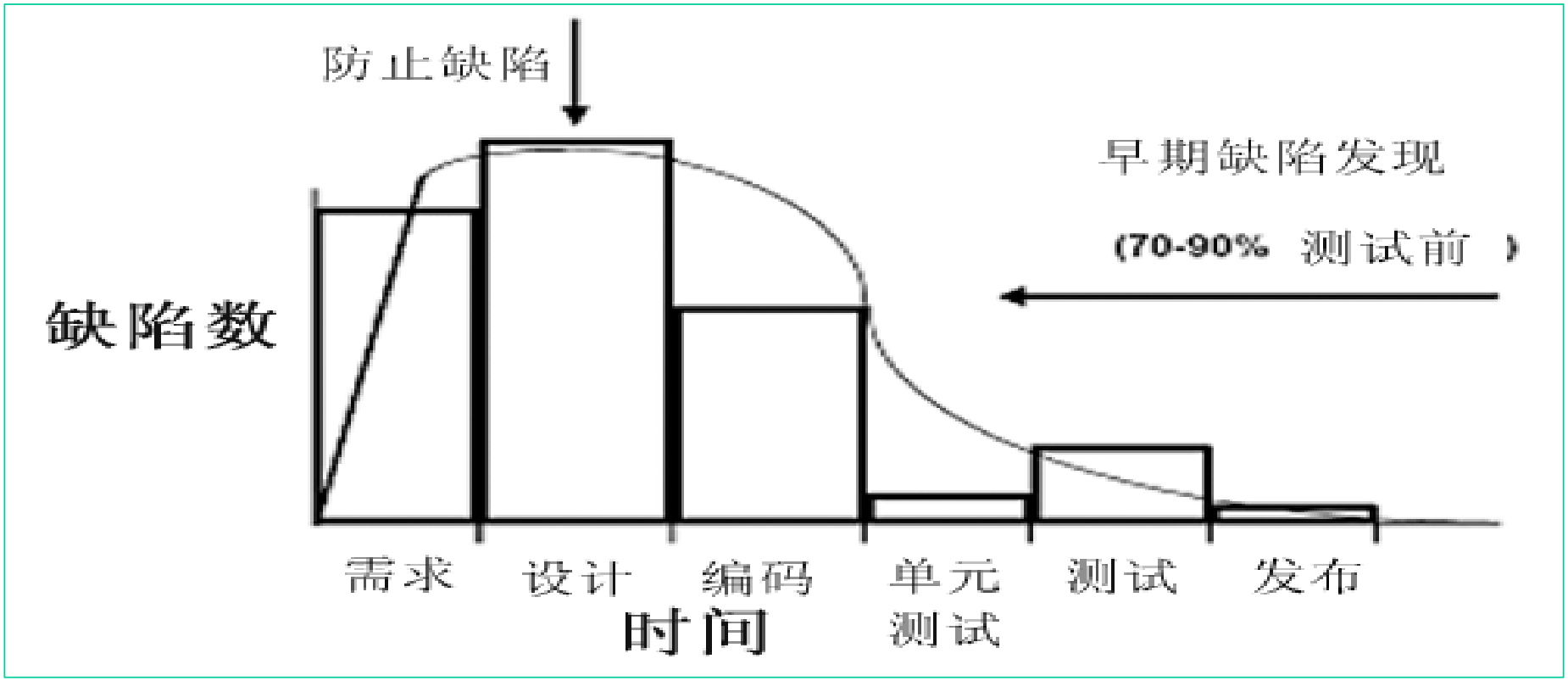
（2）时间上不协调、或不一致性所带来的问题

……



问题：为什么需求规格说明书中缺陷最多？

软件缺陷在不同阶段的分布

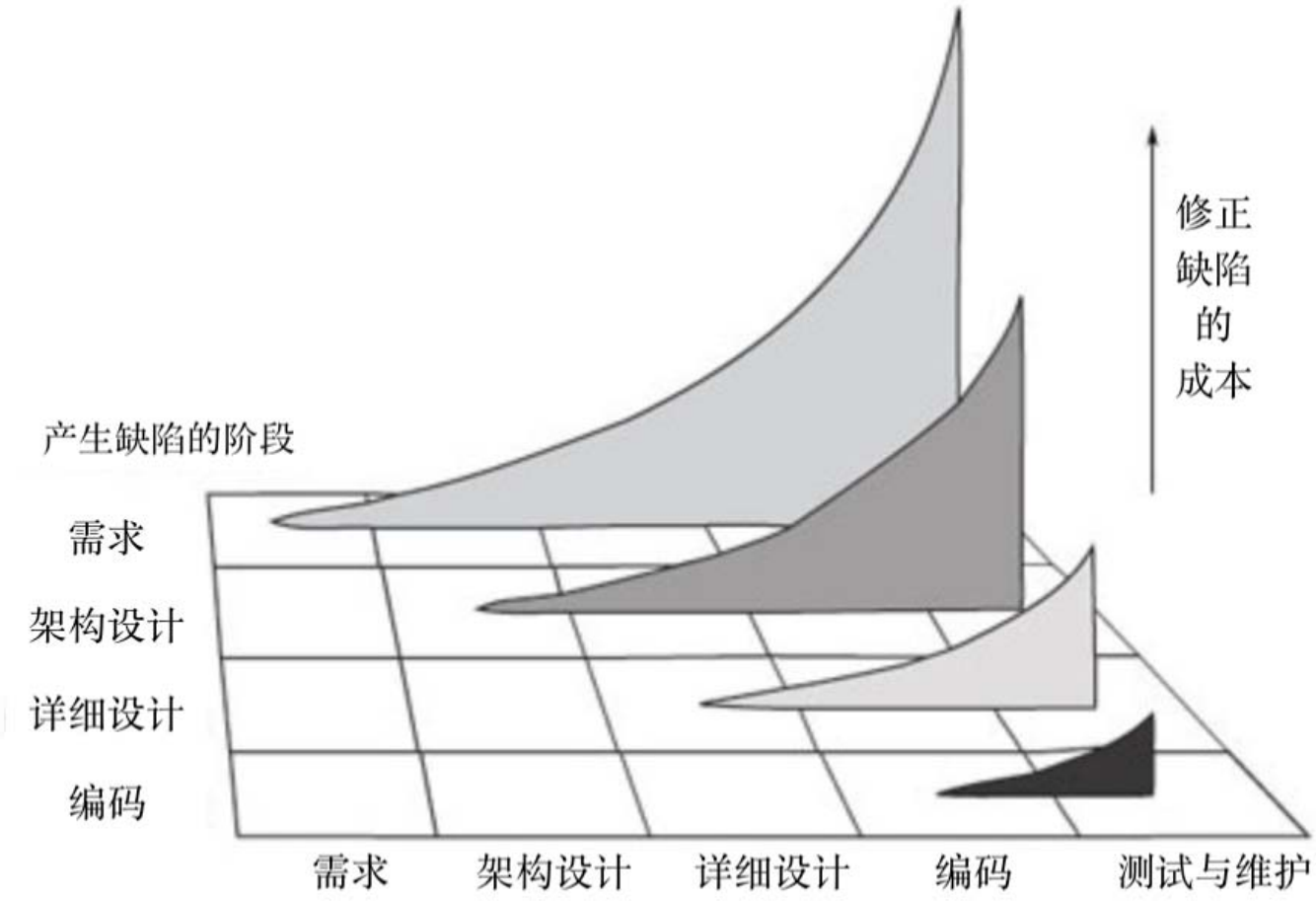


在真正的程序测试之前，通过审查、评审会可以发现更多的缺陷。

需求的缺陷会在需求评审、设计、编码、测试等过程中会逐步发现，很能在需求分析一个阶段发现

修复软件缺陷的成本

缺陷带来的成本被称为“劣质成本（COPQ）”，非线性增长，不及时处理所带来的成本很高。



软件缺陷与Bug

软件缺陷与Bug的区别虽然软件缺陷（Defect）常被称为Bug，但二者又是有区别的。

（1）软件缺陷(defect)通常是指软件设计存在的问题。

（2）Bug通常是由软件缺陷(Defect)引起的一些软件故障，Bug是软件缺陷（Defect）的一种表现形式。

（3）一个缺陷是可以引起多种Bug的。

（4）绝大部分公司将计算机的错误统称为Bug

软件故障

软件故障是指在软件运行过程中出现的一种不希望或不可接受的内部状态。软件故障是一种状态行为，是指一个实体发生障碍和毛病。

软件故障在ISO 14598软件产品评价标准中的定义是：计算机程序中不正确的步骤、过程和数据定义。

一般是指在编写代码过程中出现的错误。

“故障”指软件运行时丧失了在规定的限度内执行所需功能的能力，它是动态的，可能导致失效。故障不一定导致软件失效，软件运行可以出现故障但不出现失效，如在容错（Fault Tolerance）软件运行中容许有规定数量的故障出现而不导致失效。

对无容错的软件，故障即失效。故障是软件缺陷的外在表现。

软件故障(Fault_error_Failure)

软件在运行过程当中，运行到fault就会产生一个相应的错误，触发到一个错误的中间状态。

软件失效（Failure）Error传播到软件外部，使用户观测到失效行为。称为失效行为，当Fault导致了Error，Error传播到外面，会导致预想之外的输出，这种错误为Failure。

失效意味着系统是在运行，而且只有在执行程序过程中才会发生软件失效。

“失效”指软件运行时不能完成规定功能，它是动态的，由故障所导致。失效是软件缺陷的外在表现。

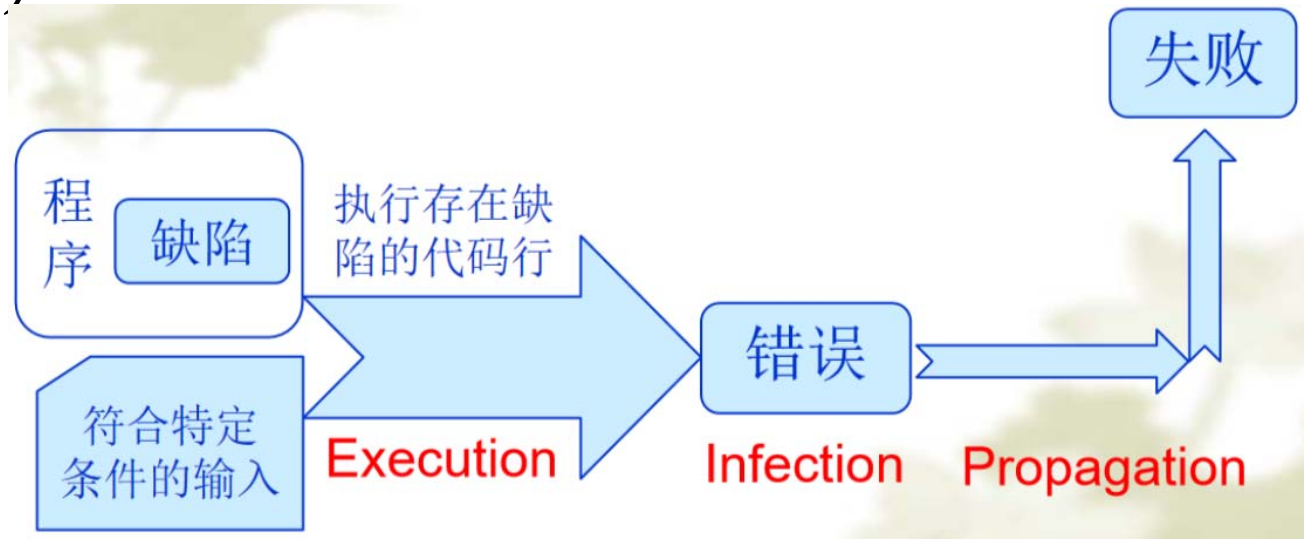
PIE模型：缺陷发现

发现一个缺陷需要满足以下三个必要条件：

1）Execution（运行）：测试必须运行到包含缺陷（Falut）的程序代码。

2）Infection（感染）：程序必须被感染出一个错误（Error）的中间状态。

3）Propagation（传播）：错误的中间状态必须传播到外部并被观察到（Failure）



PIE模型：缺陷发现

Failure也不一定会被观测到，这时候会有三种情况：

第一种：测试没有执行到Fault

第二种：感染到Error，但Error因巧合导致隐藏

第三种：没有传播到外面体现Failure

有Fault，不一定会感染Error；又Error不一定会传播出Failure。

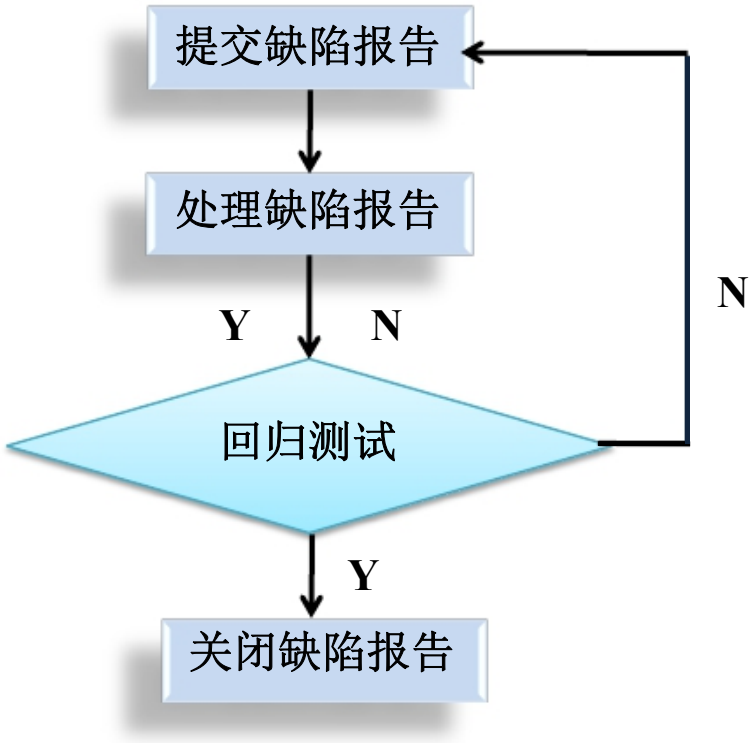
对缺陷会关心哪些问题?

- 描述?
- 是否严重?
- 是否需要修正? （优先）
- 当前状态?
- ……

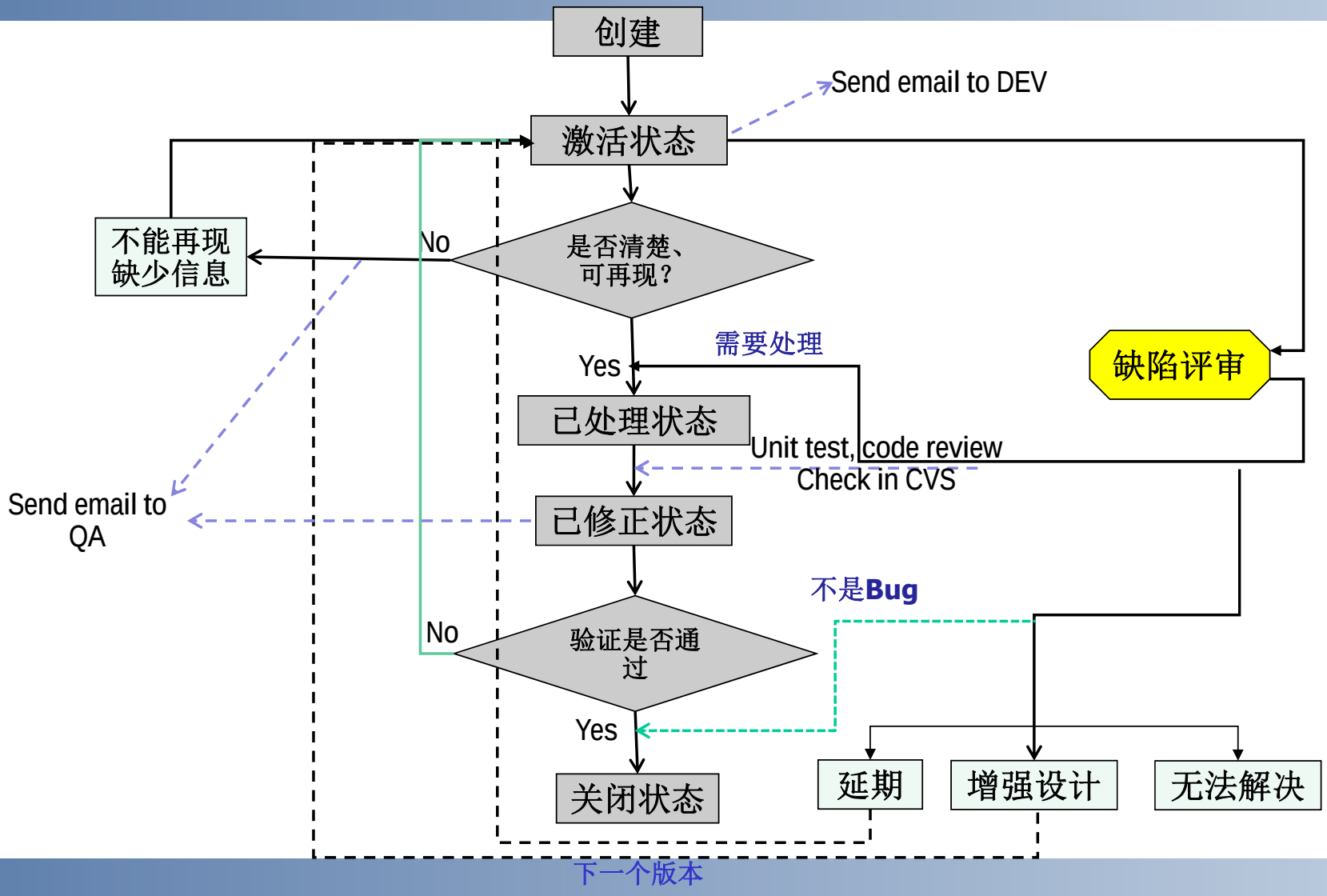


缺陷的跟踪与基本生命周期

- 测试人员
- 开发人员
- 测试人员
- 测试人员



完整的缺陷生命周期


创建
Send email to DEV
激活状态
不能再现
缺少信息
No
是否清楚、
可再现？
需要处理
Yes
缺陷评审
已处理状态
Unit test, code review
Check in CVS
Send email to
QA
已修正状态
不是Bug
No
验证是否通
过
Yes
关闭状态
延期
增强设计
无法解决
下一个版本

缺陷严重性

缺陷严重性:表示软件缺陷所造成的危害的恶劣程度

Fatal:致命的错误，造成系统或应用程序崩溃、死机、系统悬挂，或造成数据丢失、主要功能完全丧失等，

Critical:严重错误，主要指功能或特性没有实现，主要功能部分丧失，次要功能完全丧失，或致命的错误声明。

Major:主要错误，这样的缺陷虽然不影响系统的使用，但没有很好地实现功能，没达到预期效果。如提示信息不太准确，或用户界面差，操作时间长等，

Minor:一些小问题，对功能几乎没有影响，产品及属性仍可使用。一些友好的建议。

Suggestion：一些友好的建议。

严重性

Blocker 级-阻塞、倒闭	必须立即解决，畏据情况可以要求项目组立即发布版本导致开发或测试无法进行，或程序无法运行	A类
严重级	导致系统崩溃，数据丢失，严重系统资源泄漏等，关键功能缺失或错误	
关键	功能缺失或错误，界面关键信息的错误	B类
级次要级	界面非关键信息的错误	C类
易用级	使用不方便的问题	D类

缺陷优先级

问：Bug严重程度和优先级一定成正比吗？

缺陷优先级:表示修复缺陷的先后次序的指标一般优先级的划分用ABCD或数字1-4表示，A或1表示最高级别，D或4表示最低级别。

优先级别	描述	修复顺序
5-Urgent	最高优先级：在这个错误的影响下系统几乎不能使用	立即修复，停止进一步测试在产品发布之前必须修复次
4-VeryHigh	高优先级：错误对这套系统的能力产生严重的影响	在产品发布之前必须修复
3-High	中优先级：如果这个错误存在于系统中，会制约开发和测试的活动进行，如果先前没有修复它，那么需要在发布前修复它	如果时间允许应该修复
2-Medium	优先级：不会延迟发布	会在以后修正这个错误
1-Low	最低优先级 ：时间和资源允许时修正	可能会修复，但是也能不修复

缺陷修复问题

软件缺陷可能得不到修复的原因从技术角度看会有哪些（问AI）

缺陷难以复现：缺陷仅在特定环境、特定操作顺序或罕见条件下出现，开发团队无法稳定复现，难以定位根因。

技术债务过高：代码结构混乱、模块耦合度高，修复可能引发连锁反应，风险大于收益。

底层架构限制：缺陷与系统核心架构设计相关，修复需重构部分甚至全部系统，成本过高。

第三方依赖问题：缺陷由第三方库、框架或外部服务引起，需等待上游更新，或缺乏控制权。

优先级的计算

（1）可重复性 (Repeatability)

（2）可发生性 (Visibility)

（3）严重性 (Severity)

优先级=（可重复性+可发生性）* 严重性

软件缺陷--状态

缺陷状态	描述
激活或打开（ActiveorOpen）	差异！新提的缺陷，确认“提交的缺陷”，问题还没有解决，等待处理。
已修正(FixedorResolved)	已被开发人员检查、修复过的缺陷，已解决但还未被测试人员验证
关闭或非激活(ClosedorInactive)	测试人员验证后，确认缺陷不存在之后的状态。
重新打开(Reopen)	测试人员验证后，还依然存在的缺陷，等待开发人员进一步修复
推迟(Deferred)	这个软件缺陷可以在下一个版本中解决
保留(onhold)	由于技术原因或第三者软件的缺陷，开发人员不能修复的缺陷

软件缺陷--状态

缺陷状态	描述
不能重现 (Cannotduplicate)	开发不能复现这个软件缺陷，需要测试人员检查缺陷复现的步骤。
需要更多信息 (Needmoreinfor)	开发能复现这个软件缺陷，但开发人员需要一些信息，例如：缺陷的日志文件，图片等。
重复（Duplicate）	这个软件缺陷已经被其他的软件测试人员发现。
不是缺陷（Notabug）	这个问题不是软件缺陷
需要修改软件规格说明书（Specmodified）	由于软件规格说明书对软件设计的要求，软件开发人员无法修复这个软件缺陷，必须要修改软件规格说明书。

软件缺陷报告

测试活动实施过程中，测试工程师发现缺陷后，需要根据企业所定义的缺陷报告格式进行缺陷登记。

https://zhuanlan.zhihu.com/p/595499176

表 5-1　缺陷报告示例

缺陷 ID	1		
概要描述	订单查询功能查询结果日期降序排列显示功能未实现		
发现人	李四	下步处理人	张三
发现时间	2014-4-3 10:43:21	修复时间	2014-4-4 18:12:32
所属版本	OMS1.0	所属模块	订单查询
缺陷状态	Open		
缺陷严重度	High	修复优先级	
详细描述	订单查询功能处，选择起止日期后，查询结果未能以日期降序形式		

知乎 @小溪流

缺陷的其它属性

（1）缺陷标识（ID）：标记某个缺陷的唯一表示，使用数字序号

（2）缺陷类型（type）：根据缺陷的自然属性划分缺陷种类如功能、UI、性能、文档

（3）缺陷产生可能性（frequency）/可再现的概率（通常用频率）

（4）缺陷来源（source）：缺陷所在地方，如需求、设计、编码

（5）缺陷原因（cause）：造成上述错误的根本因素，以寻求软件开发流程的改进、管理水平提高。数据格式、计算错误、接口参数、变量定义与引用等

缺陷描述的基本要求

单一准确。每个报告只针对一个软件缺陷

可以再现。提供缺陷的精确操作步骤

完整统一。提供完整、前后统一的软件缺陷的步骤和信息

短小简练。包括使用关键词

特定条件。一定要注明

补充完善。继续监视其修复的全过程。

不做评价



哪点比较关键?

散漫的缺陷报告的示例

重现步骤：

- 在Windows 11上打开一个编辑文字的软件并且编辑存在文件
- 文件字体显示正常
- 我添加了图片，这些图片显示正常
- 在此之后，我创建了一个新的文档
- 在这个文档中我随意录入了大量的文字
- 在我录入这些文字之后，选择几行文字.并且通过选择Font 菜单然后选择Arial字体格式改变文字的字体。
- 有三次我重现了这个缺陷
- 我在Solaris操作系统运行这些步骤，没有任何问题。
- 我在Mac操作系统运行这些步骤，没有任何问题。

期望结果：当用户选择已录入的文字并改变文字格式的时候，文本应该显示正确的文字格式不会出现乱字符显示。

实际结果：我试着选择少量的不同的字体格式，但是只有Arial字体格式有软件缺陷，不论如何，它可能会出现在我没有测试的其它的字体格式

优秀的缺陷报告的示例

重现步骤:

(1)打开一个编辑文字的软件并且创建一个新的文档(这个文件可以录入文字);

(2)在这个文件里随意录入一两行文字;

(3)选中承入文字,选择Font菜单并选择Arial字体格式:

(4)录入文字变成了无意义的乱码,

期望结果:

当用户选择已录入的文字并改变文字格式的时候，文本应该显示正确的文字格式且不会出现乱码显示

实际结果:

它是字体格式的问题,如果在改变文字格式成Aria之前保存文件,缺陷不会出现。缺陷仅发生在Windows10并且改变文字格式成其他的字体格式，文字是显示正常的。

缺陷报告5C原则

注：测试工程师编写缺陷报告时，需遵循以下几个原则(5C)：

（1）准确（Correct）：每个组成部分描述需准确，不会引起误解。

（2）清晰（Clear）：每个组成部分描述需清晰，易于理解。

（3）简洁（Concise)：只包含必不可少的信息，不包括任何多余的内容。

（4）完整（Complete)：包含复现该缺陷的完整步骤和其他本质信息。

（5）一致（Consistent）：按照一致的格式编写全部缺陷报告。

缺陷的跟踪处理

(1)确保每个被发现的缺陷都能够被解决，“解决”的意思不一定是被修正，也可能是其他处理方式（例如，延迟到下一个版本中修正或者由于技术原因不能被修正）

(2)收集缺陷数据并根据缺陷趋势曲线识别测试处于测试过程中的哪个阶段；

(3)决定测试过程是否结束，通过缺陷趋势曲线来确定测试过程是否结束是常用并且较为有效的一种方式。

软件缺陷处理技巧

（1）确保被发现的缺陷能够被及时处理

（2）及时沟通，如reopen bug前，事先线下沟通

（3）只有测试人员有关闭缺陷的权限

（4）明确定义优先级和严重级衡量标准

（5）注释清楚、达成一致意见

（6）周期性的会诊Bug

（7）实时监控Bug Dashboard

（8）不要试图隐藏Bug

（9）收集缺陷数据，进行必要的缺陷分析

一些缺陷处理的基本操作

审阅：可以由测试管理员、项目管理员或其他人来进行，审阅缺陷报告的质量水平；

拒绝：如果审阅者决定需要对一份缺陷报告进行重大修改，应该和测试人员一起讨论，由测试人员纠正缺陷报告，然后再次提交；

完善：完整地描述了问题的特征并将其分离，那么审查者就会肯定这个报告；

分配：分配给适当的开发人员，如果不知道具体开发人员，应分配给项目开发组长，由开发组长再分配给对应的开发人员；

一些缺陷处理的基本操作 - 续

验证。缺陷的修复需要得到测试人员的验证，同时还要进行回归测试，检查这个缺陷的修复是否会引入新的问题；

重新打开。重新打开一个缺陷，需要加注释说明、电话沟通等，否则会引起“打开-修复”多个来回，造成测试人员和开发人员不必要的矛盾

关闭。只有测试人员有关闭缺陷的权限，开发人员没有这个权限。

暂缓。如果每个人都同意将确实存在的缺陷移到以后处理，应该指定下一个版本号或修改的日期。一旦新的版本开始时，这些暂缓的缺陷应该重新被打开。

软件缺陷的处理和跟踪

1、确保每个被发现的缺陷都能够被解决，“解决”的意思不一定是被修正，也可能是其他处理方式（例如，延迟到下一个版本中修正或者由于技术原因不能被修正）

2、收集缺陷数据并根据缺陷趋势曲线识别测试处于测试过程中的哪个阶段；

3、决定测试过程是否结束，通过缺陷趋势曲线来确定测试过程是否结束是常用并且较为有效的一种方式。

缺陷趋势分析

1）缺陷密度

缺陷密度是指缺陷在软件规模（组件、模块等）上的分布，如每千行代码或每个功能点的缺陷数。一般来说，发现更多缺陷的模块，隐藏的缺陷也更多，在修正缺陷时也会引入较多的错误，结果产品的质量更差。所以说，缺陷密度越低意味着产品质量越高。

（1）如果相对上一个版本，当前版本的缺陷密度没有明显变化或更低，就应该分析当前版本的测试效率是不是降低了？如果不是，意味着产品质量得到了改善；如果是，那么就需要额外的测试，还需要对开发和测试的过程进行改善。

（2）如果当前版本的缺陷密度比上一个版本高，那么就应该考虑在此之前是否为提交测试效率进行了有效的策划并在本次测试中得到实施。如果是，虽然需要开发人员更多的努力去修正缺陷，但质量还是得到更好的保证；如果没有，意味着质量恶化、质量很难得到保证。这时要保证质量，就必须延长开发周期或投入更多的资源。

软件测试各阶段的工作任务

阶段性缺陷清除率是测试缺陷密度度量的扩展，跟踪开发周期所有阶段中的缺陷，包括需求评审、设计评审、代码审查和测试等。因为大部分的编程缺陷是和设计问题有关的，进行正式评审或功能验证以增强前期过程的缺陷清除率，有助于减少缺陷的注入。

缺陷发现阶段	需求定义问题	设计问题	代码问题	阶段清除率
需求阶段—需求评审	10			需求阶段 10/32=31.25
设计阶段—设计评审	8	8		设计阶段 8/24=33.3%
编程阶段—代码评审	4	10	20	编程阶段 20/40 =50%
系统测试	8	5	15	
验收测试	2	1	5	
合计	32	24	40	

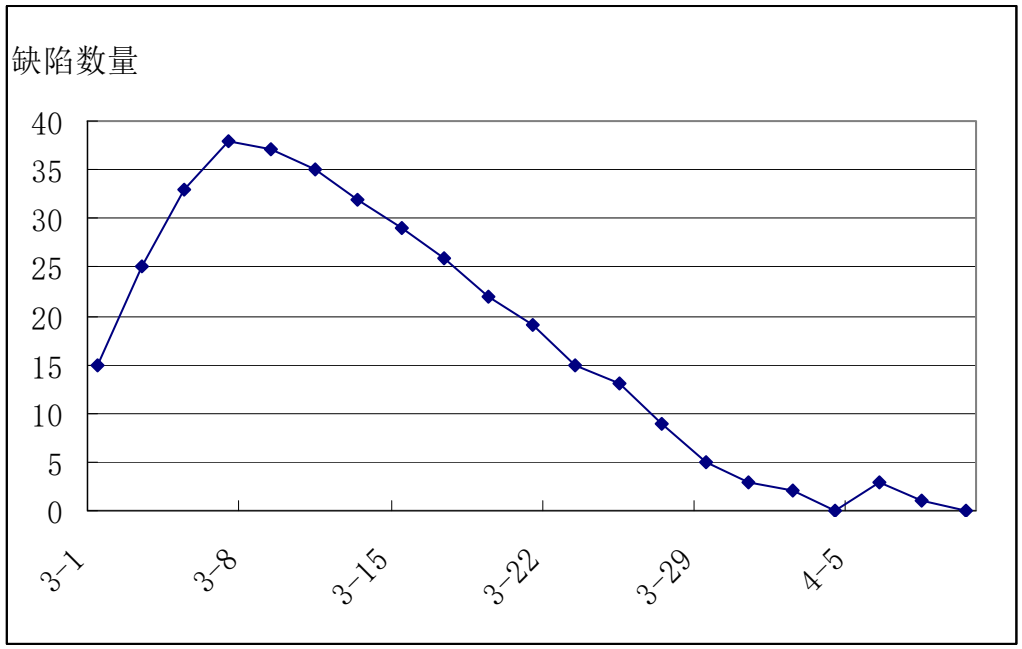
缺陷趋势分析

监控（打开/关闭/已修正的）缺陷随时间的变化

（1）产品开发质量情况取决于累积打开/关闭曲线的趋势。

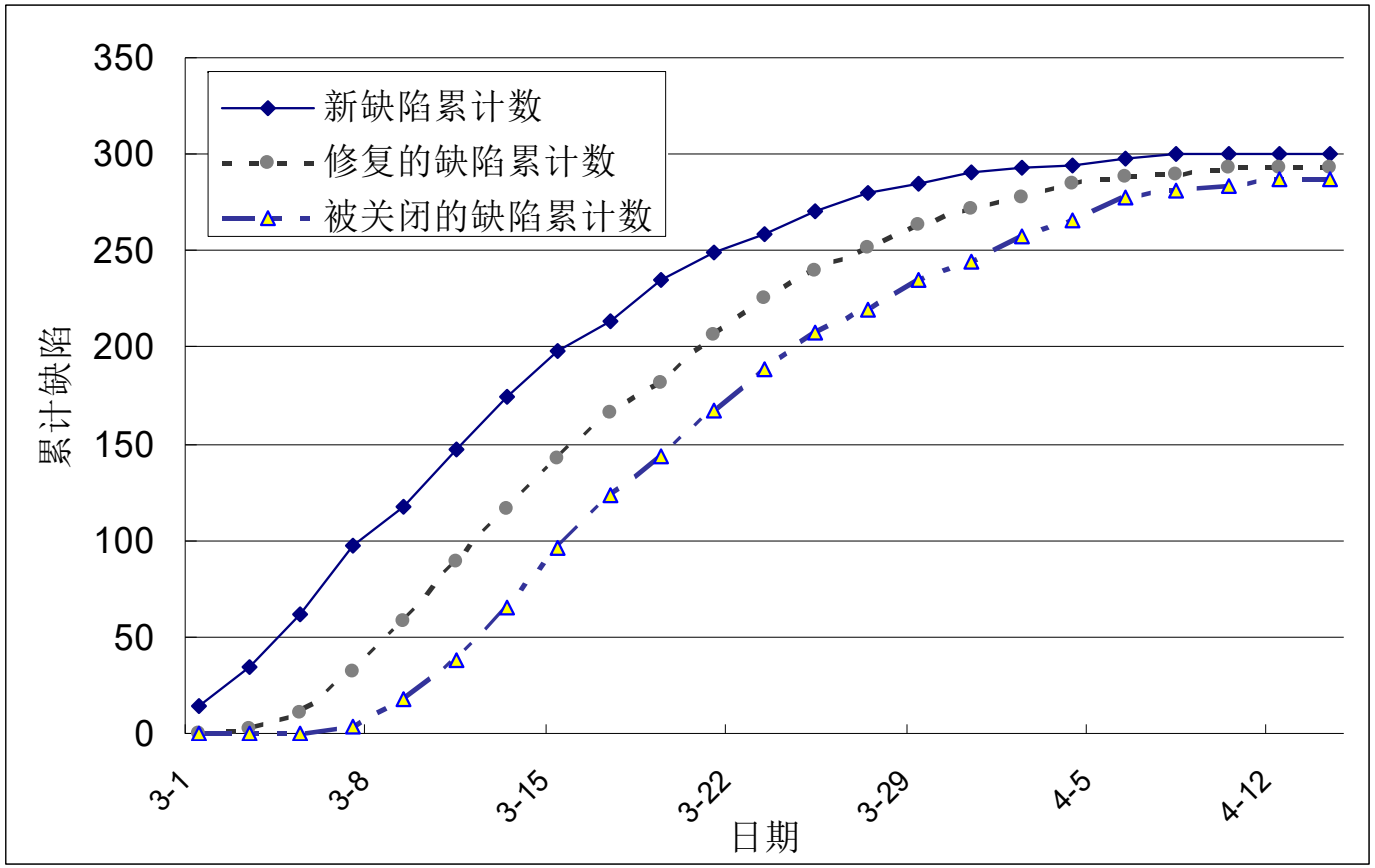
（2）项目进度取决于累积关闭/打开曲线起点的时间差。

（3）开发人员、测试人员的工作进度、效率也能得到反映



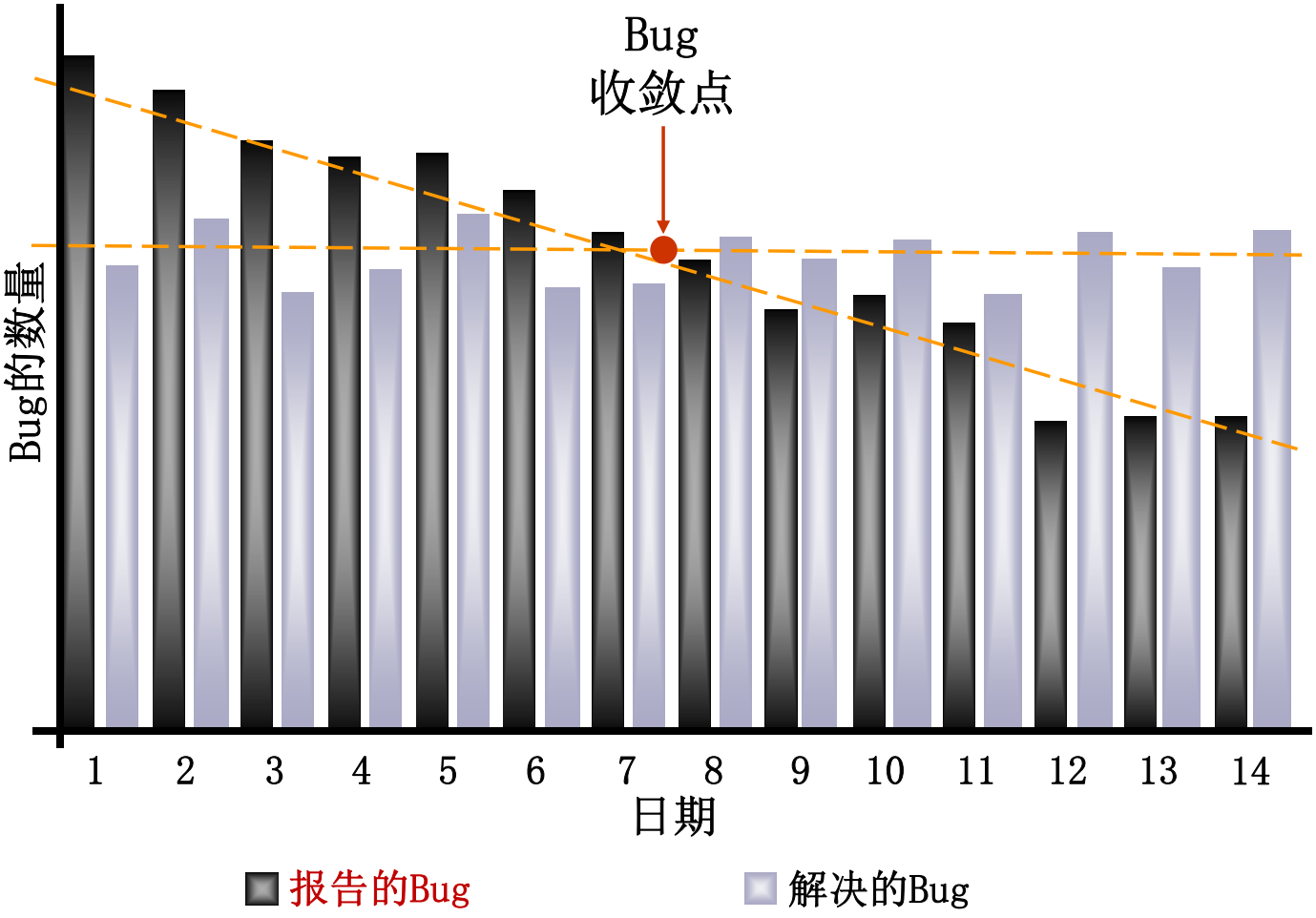
缺陷理想趋势图

累积缺陷趋势分析



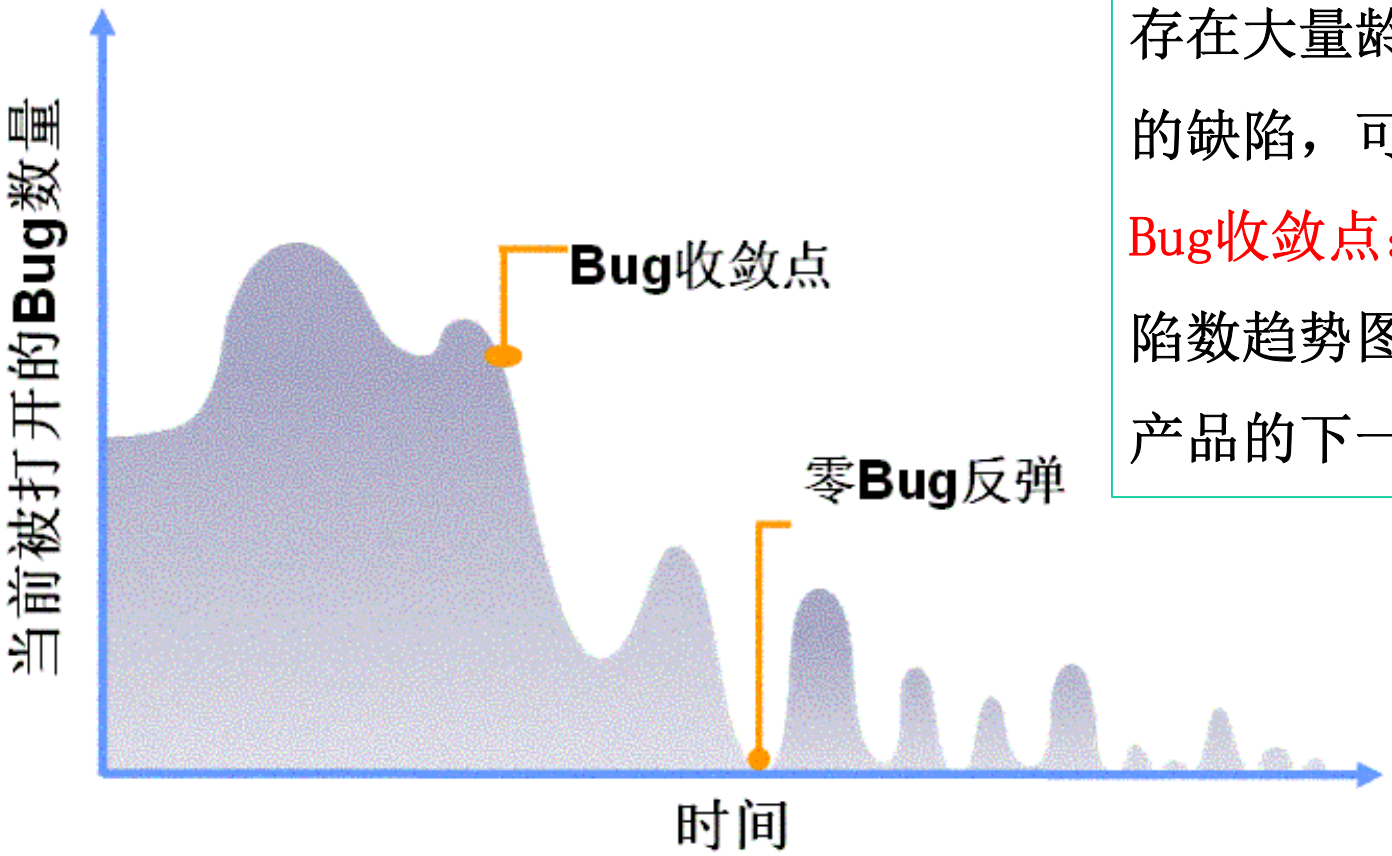
累积缺陷趋势分析

收敛点



Bug收敛点：利用发现的缺陷数和关闭的缺陷数趋势图，找出缺陷的收敛点，来预测产品的下一个阶段计划。

不断反弹之中收敛



存在大量龄期较长的、未验证的、已修正的缺陷，可能表明没有充足的测试人员。

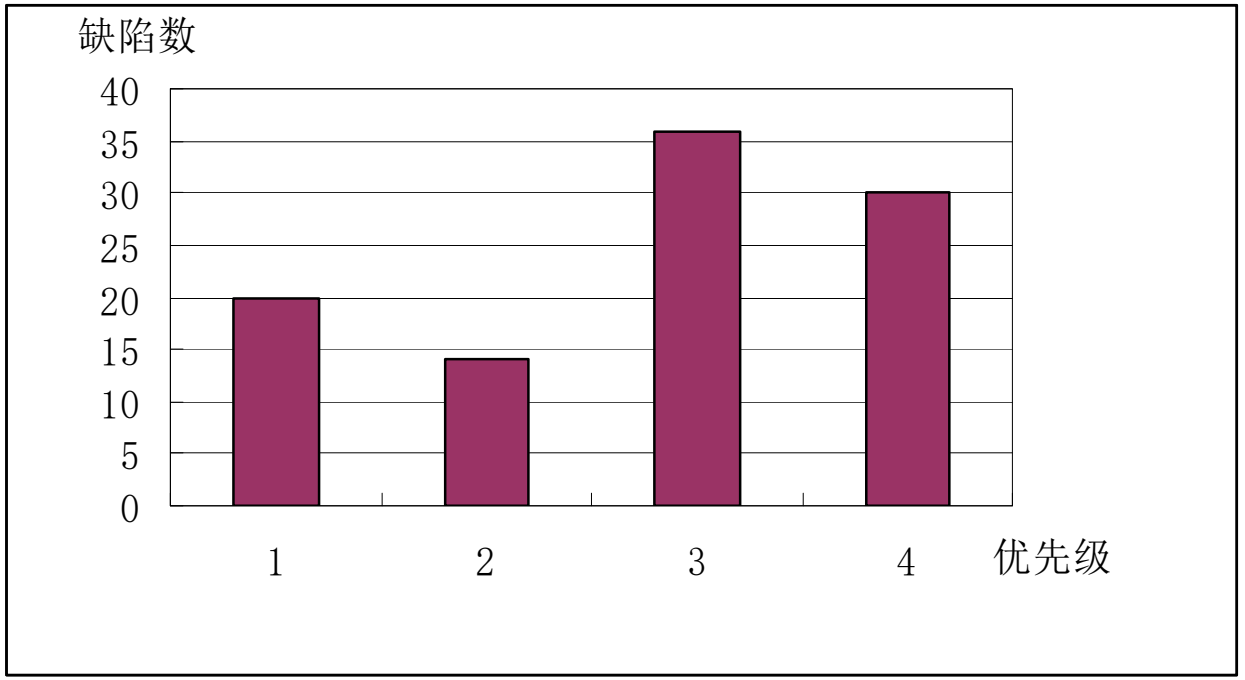
Bug收敛点：利用发现的缺陷树和关闭的缺陷数趋势图，找出缺陷的收敛点，来预测产品的下一个阶段计划。

零bug反弹：没有激活状态缺陷的第一个时间被定义为零bug反弹点从这一时刻开始，产品进入稳定期。

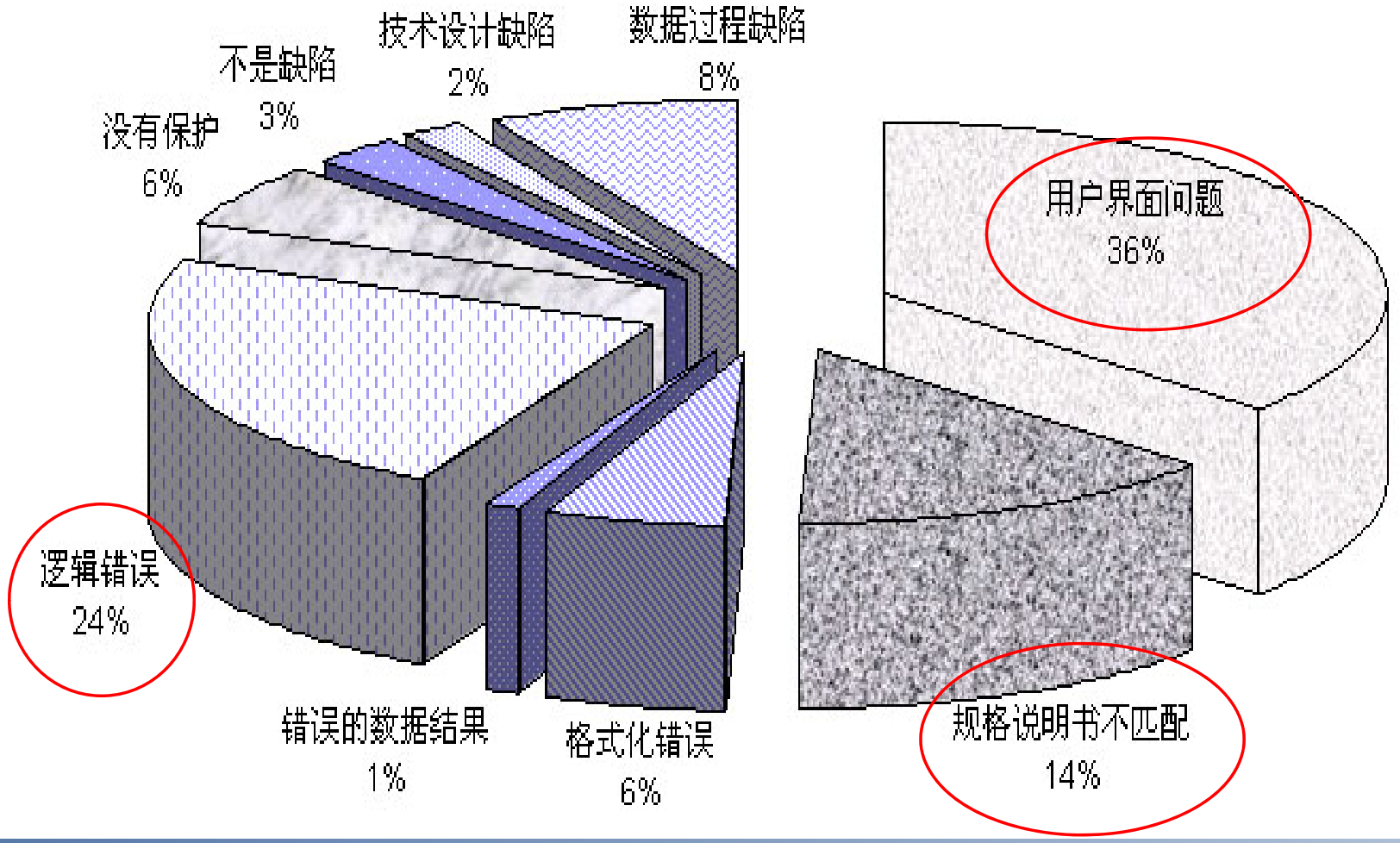
缺陷分布分析

缺陷分布报告，缺陷数量与缺陷属性的函数。如测试需求和缺陷状态、严重性的分布情况等。

P1<p2<p3



示例：缺陷分布分析


技术设计缺陷
2%
数据过程缺陷
8%
不是缺陷
3%
没有保护
6%
逻辑错误
24%
错误的数据结果
1%
格式化错误
6%
用户界面问题
36%
规格说明书不匹配
14%

缺陷报告

缺陷分布报告，允许将缺陷计数作为一个或多个缺陷参数的函数来显示，生成缺陷数量与缺陷属性的函数。如测试需求和缺陷状态、严重性的分布情况等。

缺陷趋势报告，按各种状态将缺陷计数作为时间的函数显示。趋势报告可以是累计的，也可以是非累计的；

缺陷年龄报告，显示缺陷处于活动状态的时间，展示一个缺陷处于某种状态的时间长短，从而了解处理这些缺陷的进度情况。

测试结果进度报告，展示测试过程在被测应用的几个版本中的执行结果以及测试周期

软件度量的分类与过程

分类：软件过程度量、项目度量、产品质量度量

- **识别目标：**分析出度量的工作目标和列表，并由管理者审核确认
- **定义度量过程：**定义其收集要素、收集过程、分析、反馈过程、IT支持体系，为具体的收集活动、分析、反馈活动和IT设备、工具开发提供指导
- **搜集数据：**应用IT工具进行数据收集工作，并按指定的方式审查和存储
- **数据分析与反馈：**根据数据收集结果，按照已定义的分析方法进行数据分析，完成规定格式的图表，进行反馈
- **过程改进：**根据度量的分析报告，管理者基于度量数据做出决策

基于缺陷的质量度量

代码的缺陷密度：每千行代码（KLOC）的缺陷数（Bug# /KLOC）

缺陷清除率：各个阶段的缺陷清除率和总的缺陷清除率

缺陷逃逸率：未在研发阶段发现的缺陷意味着逃逸出去，即线上发现的缺陷（生产环境发现的缺陷），缺陷逃逸率=线上发现的缺陷数/（线上发现的缺陷数+研发环境下发现的缺陷数）；

缺陷趋势是否良好，是否收敛

缺陷分布是否符合正态分布？是否过于集中在1-2个模块？

经典的种子公式

$$\frac{\text{已测试出的种子Bug（s）}}{\text{所有的种子Bug（S）}} = \frac{\text{已测试出的非种子Bug（n）}}{\text{全部的非种子Bug（N）}}$$

则可以推出程序的总Bug数为：$N = S * n / s$

其中n是所进行实际测试时发现的Bug总数。如果 n = N，说明所有的Bug已找出来，说明做的测试足够充分。

这种测试是否充分，可以用一个信心指数来表示，即用一个百分比表示，值越大，说明对产品质量的信心越高，最大值为1。

$$C \begin{cases} = 1 & if\ n>N \\ = S/(S-N+1), & if\ n<=N \end{cases}$$

基于缺陷清除率的估算方法

F：描述软件规模用的功能点；D1为在软件开发过程中发现的所有缺陷数；D2为软件发布后发现的缺陷数；

D为发现的总缺陷数。D=D1+D2

- 质量 = D2/F
- 缺陷注入率 = D/F
- 整体缺陷清除率=D1/D

缺陷源	潜在缺陷	清除效率(%)	被交付的缺陷
需求报告	1.00	77	0.23
设计	1.25	85	0.19
编码	1.75	95	0.09
文档	0.60	80	0.12
错误修改	0.40	70	0.12
合计	5.00	85	0.75

质量度量的统计方法

缺陷原因归纳（80/20规律：80%缺陷由20%主要因素造成，20%缺陷由其他80%次要因素造成）

- 说明不完整或说明错误（IES）
- 与客户交流不够所产生的误解(MCC)
- 故意与说明偏离（IDS）
- 违反编程标准（VPS）
- 数据表示有错（EDR）
- 模块接口不一致（IMI）
- 设计逻辑有错（EDL）
- 不完整或错误的测试（IET）
- 不准确或不完整的文档（IID）
- 将设计翻译成程序设计语言中的错误(PLT)
- 不清晰或不一致的人机界面（HCI）
- 杂项（MIS）

质量度量的统计方法 – 续

总计(E_i)			严重(S_i)		一般(M_i)		微小(T_i)	
错误	数量	百分比	数量	百分比	数量	百分比	数量	百分比
IES	296	**22.3%**	55	**28.2%**	95	18.6%	146	23.4%
MCC	204	**15.3%**	18	9.2%	87	17.0%	99	15.9%
IDS	64	4.8%	2	1.0%	31	6.1%	31	5.0%
VPS	34	2.6%	1	0.5%	19	3.7%	14	2.2%
EDR	182	**13.7%**	38	**19.5%**	90	17.6%	54	8.7%
IMI	82	6.2%	14	7.2%	21	4.1%	47	7.5%
EDL	64	4.8%	20	**10.3%**	17	3.3%	27	4.3%
IET	140	**10.5%**	17	8.7%	51	10.0%	72	11.6%
IID	54	4.1%	3	1.5%	28	5.5%	23	3.7%
PLT	87	6.5%	22	**11.3%**	26	5.1%	39	6.3%
HCI	42	3.2%	4	2.1%	27	5.3%	11	1.8%
MIS	81	6.1%	1	0.5%	20	3.9%	60	9.6%
总计	1330	100%	195	100%	512	100%	623	100%

质量度量计算

Si-严重错误数

Mi-一般错误数

Ti-微小错误数

阶段错误度量

$$PI_i = W_s (S_i/E_i) + W_m(M_i/E_i) + W_i (T_i/E_i)$$

总体质量度量

$$E_p = \Sigma (i \times PI_i)/P_s$$

Ei-软件工程中的第i步中发现的错误

Pi-第i步的产品规模（LOC、设计说明、文档页数）

其中 i = 1, 2, 3, 4, 5 代表需求分析、设计、编程、测试、发布

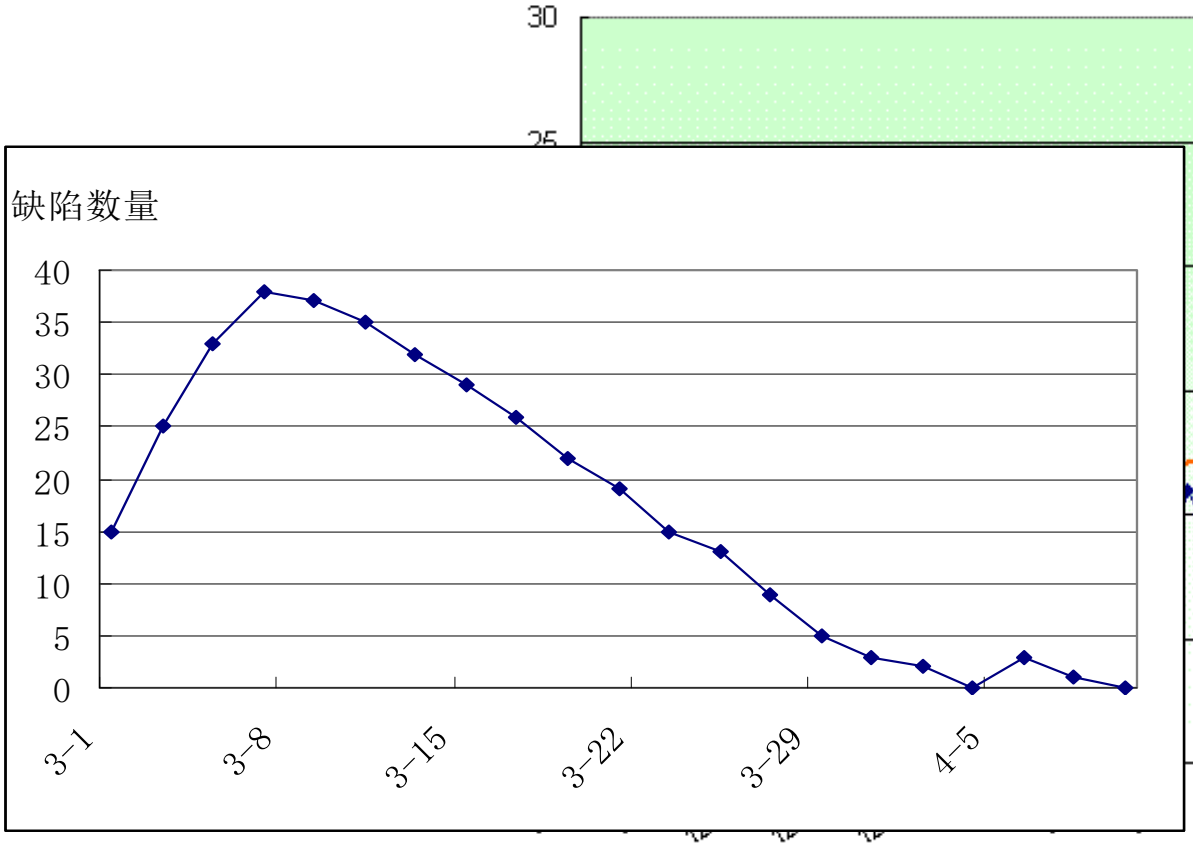
(i= 1, 2, 5, 10, 100)； $W_s, W_m, W_i = 0.6, 0.3, 0.1$

测试过程的评估

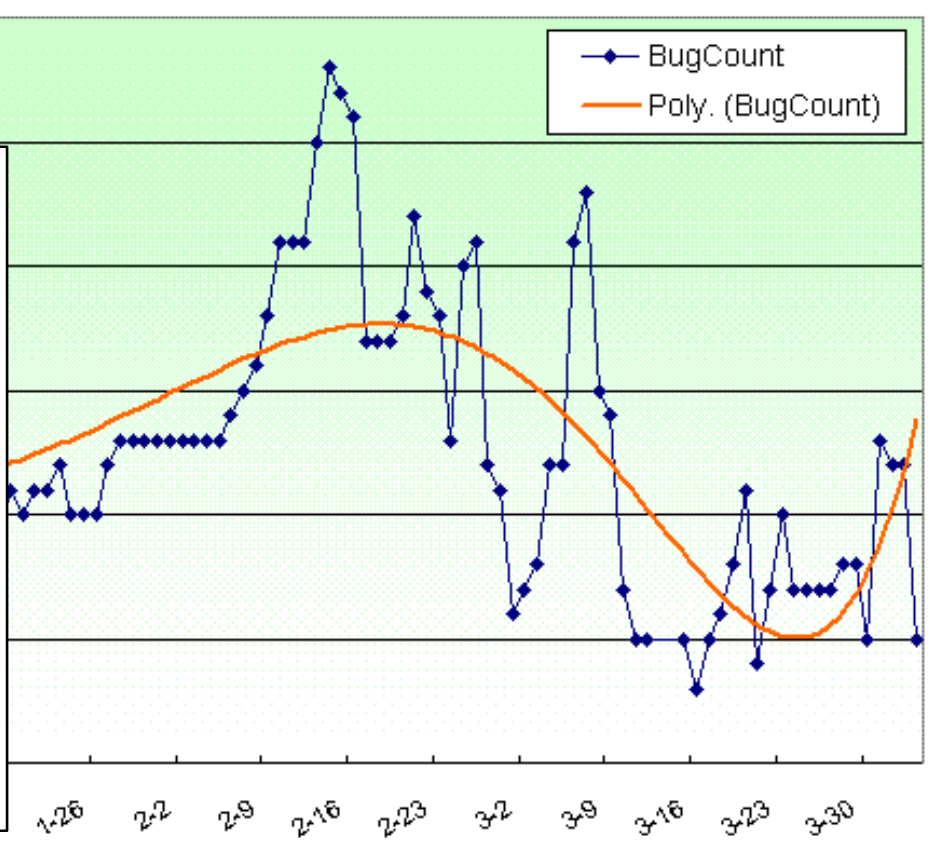
测试过程评审，结合测试计划来进行评审。

- 单元测试采用工具方法，代码覆盖率是否达到设定目标？
- 集成测试是否验证了所有接口及参数
- 系统测试是否包含了性能、兼容性、安全性等各项测试？如果执行了，又是怎么进行的、结果如何？
- 是否完成了测试计划所要求的各项测试内容？
- 测试用例是否经过开发人员、产品经理的严格评审？
- 需要执行的测试用例是否百分之百地完成了？
- 所有严重的Bug都修正了？

测试过程的风险与评估



缺陷理想趋势图



出现异常的缺陷随时间分布图

测试充分性的评估

测试结果分析的一项重要工作就是测试充分性的评估，通过了解测试覆盖率的值，可以知道测试是否充分、测试能否结束

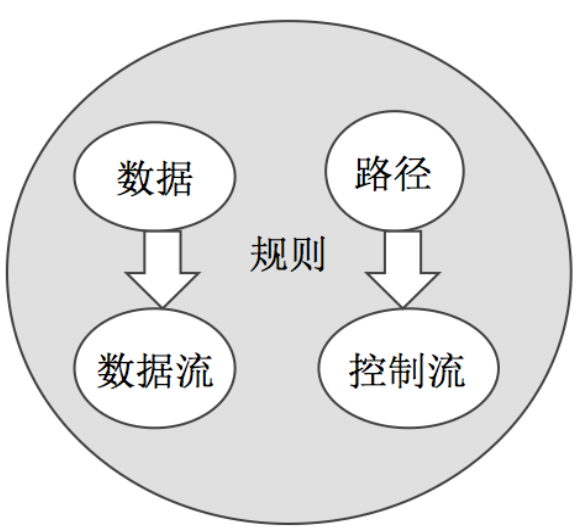
测试覆盖率是用来衡量测试完成程度、或评估测试活动覆盖产品代码的一种量化的结果，评估测试工作的质量，也是产品代码质量的间接度量方法

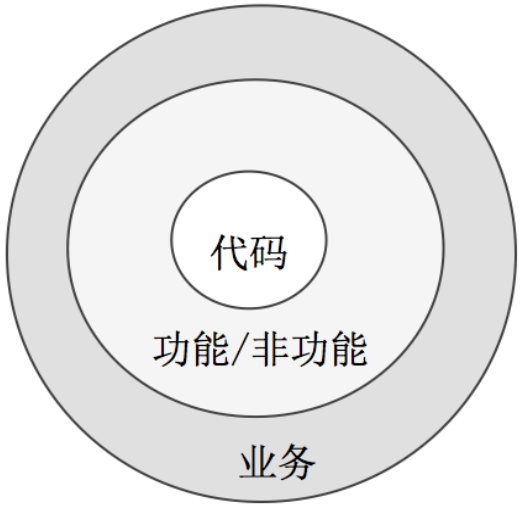
测试覆盖率可以看作由业务需求、系统特性功能／非功能特性和代码等多个层次的等构成

测试覆盖率分析

测试覆盖率：用来衡量测试完成程度或者评估测试活动覆盖产品代码的一种量化结果，是评估测试工作的质量也是产品代码质量的间接度量方法（测试过程中已验证的区域或集合）“要求被测的总区域与集合”比值。

测试覆盖率可以看作由业务需求、系统特性功能 / 非功能特性和代码等多个层次的等构成





完成测试报告

在国家标准GB/T 17544－1998（附录E）对测试报告有了具体要求，对测试纪录、结果如实汇总分析，报告出来

产品标识；

用于测试的计算机系统

使用的文档及其标识

产品描述、用户文档、程序和数据的测试结果；

与要求不符的清单；

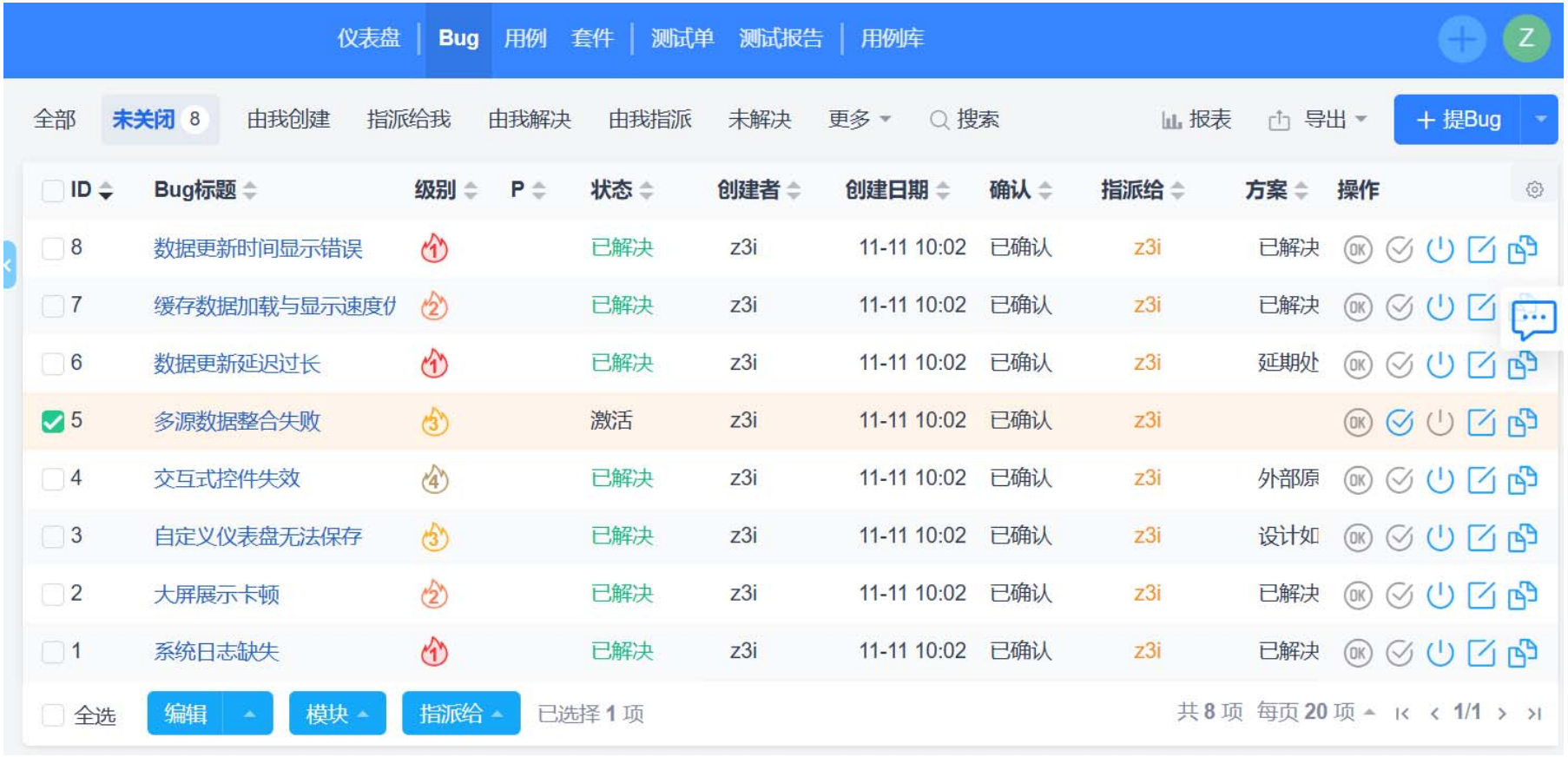
针对建议的要求不符的清单，产品未作符合性测试的说明；

测试结束日期。

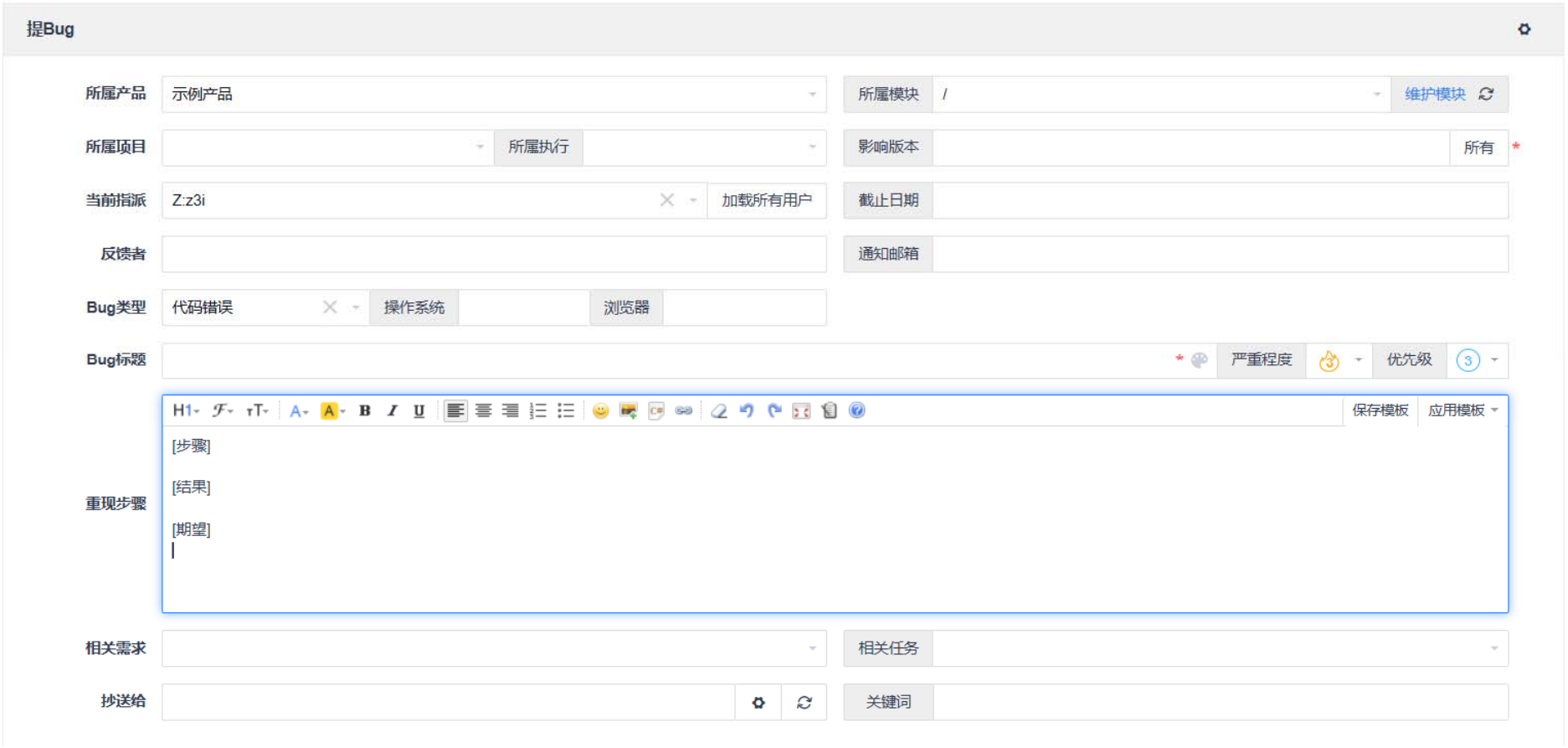
测试报告


测试 > 示例产品
仪表盘 Bug 用例 套件 测试单 测试报告
返回
1
2024-11-11 PROJECT#1 BI大屏性能优化与实时数据更新项目 测试报告
基本信息 测试范围 测试轮次 关联的用例 遗留的Bug 报表 总结 历史记录 更多功能
起止时间 2024-11-04 ~ 2024-11-11
负责人 z3i
参与人员 z3i
概况 本页共 0 个研发需求，预计 0 个工时，用例覆盖率 0%。
共测试了1个版本。共有5个用例，共执行5个用例，产生了5个结果，失败的用例有1个。
共发现0个Bug ，遗留0个Bug 。重新激活0个Bug 。用例执行产生0个Bug 。有效Bug率（方案为已
例创建的Bug / 发现Bug数）：0%
附件

禅道缺陷管理


仪表盘 Bug 用例 套件 测试单 测试报告 用例库
全部 未关闭 8 由我创建 指派给我 由我解决 由我指派 未解决 更多 搜索 报表 导出 + 提Bug
ID Bug标题 级别 P 状态 创建者 创建日期 确认 指派给 方案 操作
8 数据更新时间显示错误 已解决 z3i 11-11 10:02 已确认 z3i 已解决
7 缓存数据加载与显示速度仍 已解决 z3i 11-11 10:02 已确认 z3i 已解决
6 数据更新延迟过长 已解决 z3i 11-11 10:02 已确认 z3i 延期处
5 多源数据整合失败 激活 z3i 11-11 10:02 已确认 z3i
4 交互式控件失效 已解决 z3i 11-11 10:02 已确认 z3i 外部原
3 自定义仪表盘无法保存 已解决 z3i 11-11 10:02 已确认 z3i 设计如
2 大屏展示卡顿 已解决 z3i 11-11 10:02 已确认 z3i 已解决
1 系统日志缺失 已解决 z3i 11-11 10:02 已确认 z3i 已解决
全选 编辑 模块 指派给 已选择 1 项 共 8 项 每页 20 项 1/1

禅道缺陷管理


提Bug
所属产品
示例产品
所属模块
/
维护模块
所属项目
所属执行
影响版本
所有
当前指派
Z:z3i
加载所有用户
截止日期
反馈者
通知邮箱
Bug类型
代码错误
操作系统
浏览器
Bug标题
严重程度
优先级
保存模板
应用模板
重现步骤
[步骤]
[结果]
[期望]
相关需求
相关任务
抄送给
关键词